

Module 6 Lab:

CodeMR Tool – Use Case and Scenario-Based Coverage

JR Clark (002035589)

NEU CSYE6255 (Summer 2026)

Professor Omar Badreldin / TA Karthik Kashyap

PART 1: Tool Installation & Project Setup

CodeMR was installed in a pre-existing Eclipse IDE installation per the Lab instructions (Appendix 1, Figures 1-13). After successful setup, running the tool on larger code bases resulted in a popup (Figure 14) notifying me of limits of the free tier. I downloaded the trial license from the website and it returned an error (Figure 15). The CodeMR team responded to my request for a new trial license quickly, followed by successful activation and processing five existing INFO5100 projects.

PART 2: CodeMR Analysis

CodeMR analysis was run on five projects via “Extract Model” in Eclipse IDE, and CodeMR Graphs were generated for each project. Key metrics are summarized below (see also Appendix 3, Table 1):

Info5100-University-Example demonstrated well-distributed complexity: 76.4% of classes rated low complexity (C3), 14.1% medium, and 9.5% higher complexity. Coupling was low, cohesion was high, and size metrics were low-to-medium. The 12-package structure reflects deliberate separation of concerns across functional domains (student, faculty, course, department, etc.). **Assignment02** presented a markedly different profile. Despite having fewer classes (14 vs. 39), it contained over three times the LOC (1,826 vs. 581), indicating significantly larger, more complex individual classes. The complexity chart was dominated by yellow and orange, with the majority of classes rated medium-high or high. The 148 external classes and 26 external packages signal heavy reliance on external libraries relative to the project's own code volume. With only 2 packages, the project lacks the structural separation visible in the **University Example**.

RangePricing exhibits 71.5% “medium-high” complexity across 62 classes, 18 packages and 2430 LOC. **Midterm** has the highest LOC of all five codebases, and 80% medium-high complexity, but nearly 12% high to very high, with 3 problematic classes. This increase in complexity is reflected in the treemap chart (Figure 44) which is nearly all yellow, with one each orange and one red block. **Final** is the most red set of charts with 72.9% high to very-high complexity. The bubble chart (Figure 48) and treemap (Figure 49) indicate a structure that is overly concentrated and may require restructuring. **Final** also has the highest number of problematic classes (23) but none are “highly problematic.” These 23 flagged classes represent meaningful technical debt; functional code that carries elevated risk of defects and increased cost of change as the project evolves.

PART 3: CodeMR Visualization & Code Quality Assessment

CodeMR generates five primary visualizations based on the following parameters: Complexity, Coupling and Cohesion, collectively as “C3” in the reports:

Complexity: “Higher levels of complexity in software increase the risk of unintentionally interfering with interactions and so increases the chance of introducing defects when making changes” (CodeMR, 2021). Complexity also makes it more difficult to test, review, and maintain your code, resulting in more bugs, slower onboarding, and higher technical debt (Tanner, 2025). See Appendix 9 for typical thresholds of complexity.

Cohesion: “high cohesion is associated with several desirable traits of software including robustness, reliability, reusability, and understandability.” (CodeMR, 2021)

Coupling: more children mean greater risk in breaking behaviors, requiring more testing and making it more difficult to modify a class (CodeMR, 2021). High coupling and low

cohesion, for example, may require refactoring before modification. See Appendix 10 for coupling categories and what they mean.

Appendix 3, Table 2 shows a shift from low complexity to high complexity across all metrics and in the complexity of the “package dependencies” graphic. Three of the projects have a colorful interweaving of dependencies, but **Assignment02** and **Final** are really dominated by a single color indicating excessive dependence on a few packages. Dense inter-package coupling increases the risk that changes in one package cascade unexpectedly into others, while flat single-package dominance may reflect structural consolidation rather than intentional design.

However, the graphs are not a substitute for control flow graphs or execution traces. CodeMR's cyclomatic complexity score approximates the number of independent paths through a method, the same concept underlying CFG-based path coverage, but without explicit path enumeration, it cannot identify unreachable branches or verify that all decision paths have been exercised by tests.

Recommendations: Although **Assignment02** is relatively small, it could be improved by increasing package modularity. **Midterm**'s 3 problematic classes should be prioritized for refactoring before the codebase scales further. **Final**'s treemap (Figure 49) and package dependencies (Figure 50) show over-concentration in the panels packages, and the 588 external classes signaling vulnerability to dependency versioning issues. Ideally, this codebase would reorganize large monolithic classes into smaller modules, and reduce dependency concentration in the panels package to lower architectural risk.

PART 4: Critical Evaluation & Analysis

CodeMR provides immediate visual insight into code quality without requiring test execution or runtime instrumentation. The combination of donut charts, sunburst, treemap, and

chord diagram gives a rapid multi-perspective view of a codebase that would take hours to assess manually. The HTML report format is shareable and accessible without an Eclipse installation. For teams performing code reviews or onboarding new contributors, CodeMR's visual output communicates structural complexity in a way that raw metrics tables cannot.¹

The comparative analysis in this lab demonstrated CodeMR's ability to distinguish meaningfully between a well-architected project and a monolithic one, even when both report zero problematic classes. The metrics tell a nuanced story: Assignment02 is not broken, but its structure would present real maintainability challenges as the codebase grows.

Several limitations emerged during this lab, including the free tier limits and the trial license activation failure, that exclude any real-world project of meaningful scale and limit accessibility for students on a timeline.

CodeMR measures structural attributes (complexity, coupling, cohesion, size) but does not evaluate test coverage, runtime behavior, security vulnerabilities, or code correctness. A class can score Low complexity and still contain a critical logic error. CodeMR results must be interpreted alongside test results and code review, not used as a standalone quality indicator.

Recommendation: CodeMR is best suited for early architectural review on small-to-medium Java projects where visual communication of code quality is a priority. It is particularly useful for academic settings, code review discussions, and pre-refactoring analysis. For larger projects or teams requiring security analysis, test coverage integration, or CI/CD pipeline enforcement, tools like SonarQube offer broader capabilities with comparable visualization at enterprise scale.

¹ "The debt treemap sizes each method by code volume and colors it by this score. The biggest, reddest blocks are the highest-return refactoring targets. An executive doesn't need to know what cyclomatic complexity means. They point at the red block and say fix that one first." (Schneider, 2026).

APPENDIX 1: Tool Installation & Project Setup

Figure 1. Confirming Java Development Kit Installation

```
blackcatranch-m3 — -zsh — 80x24

[blackcatranch-m3@JRC-M3 ~ % java -version
openjdk version "25.0.3" 2026-04-21 LTS
OpenJDK Runtime Environment Temurin-25.0.3+9 (build 25.0.3+9-LTS)
OpenJDK 64-Bit Server VM Temurin-25.0.3+9 (build 25.0.3+9-LTS, mixed mode, shari
ng)
blackcatranch-m3@JRC-M3 ~ % ]
```

Figure 2. Confirming Eclipse IDE Installation & Access to Java Project

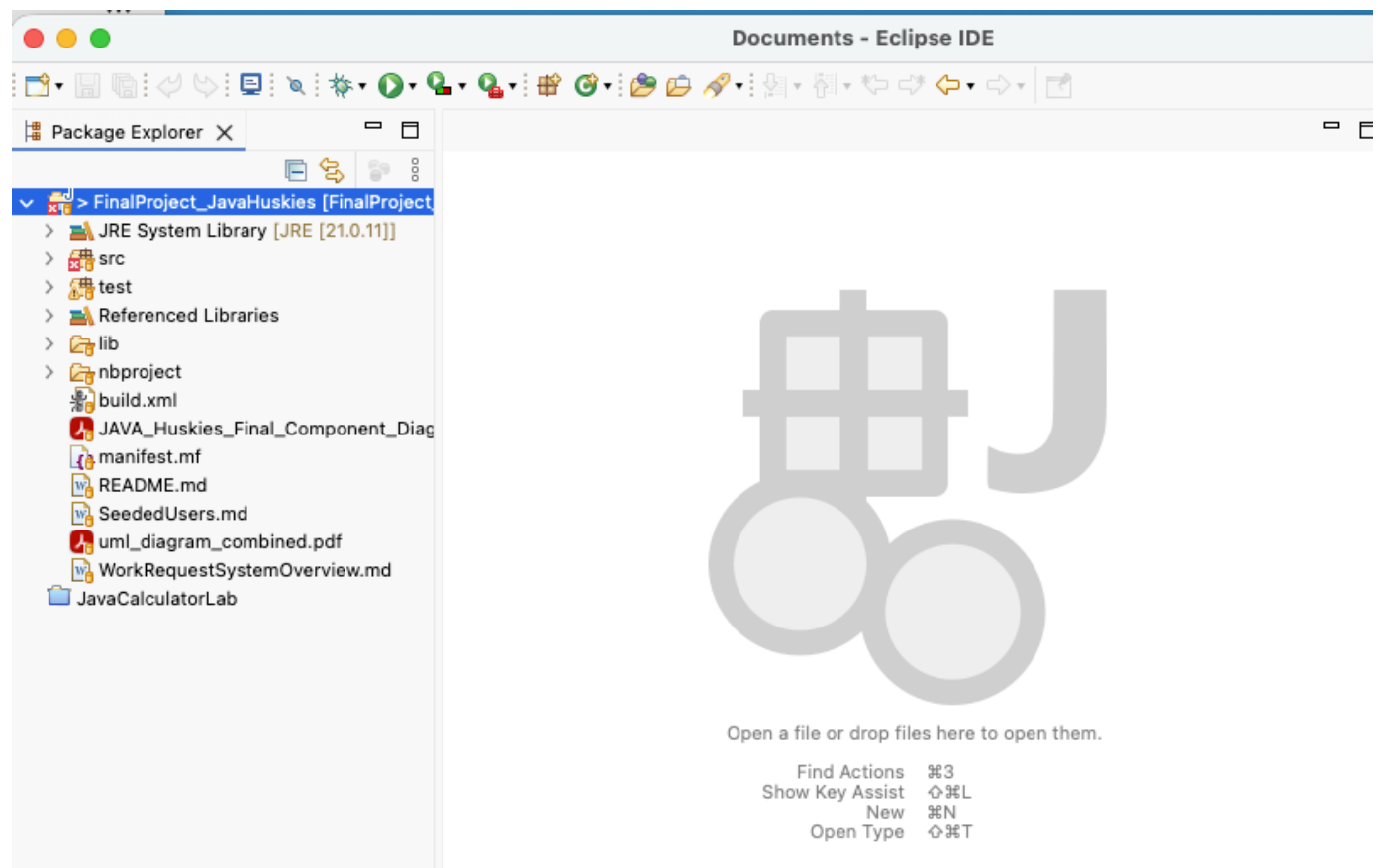


Figure 3. Add the CodeMR Update Site

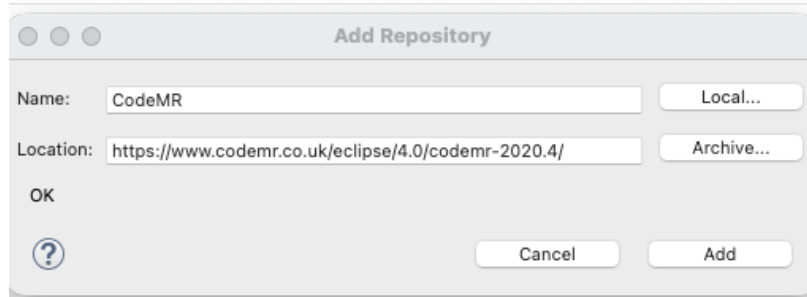


Figure 4. Select & Install CodeMR Component

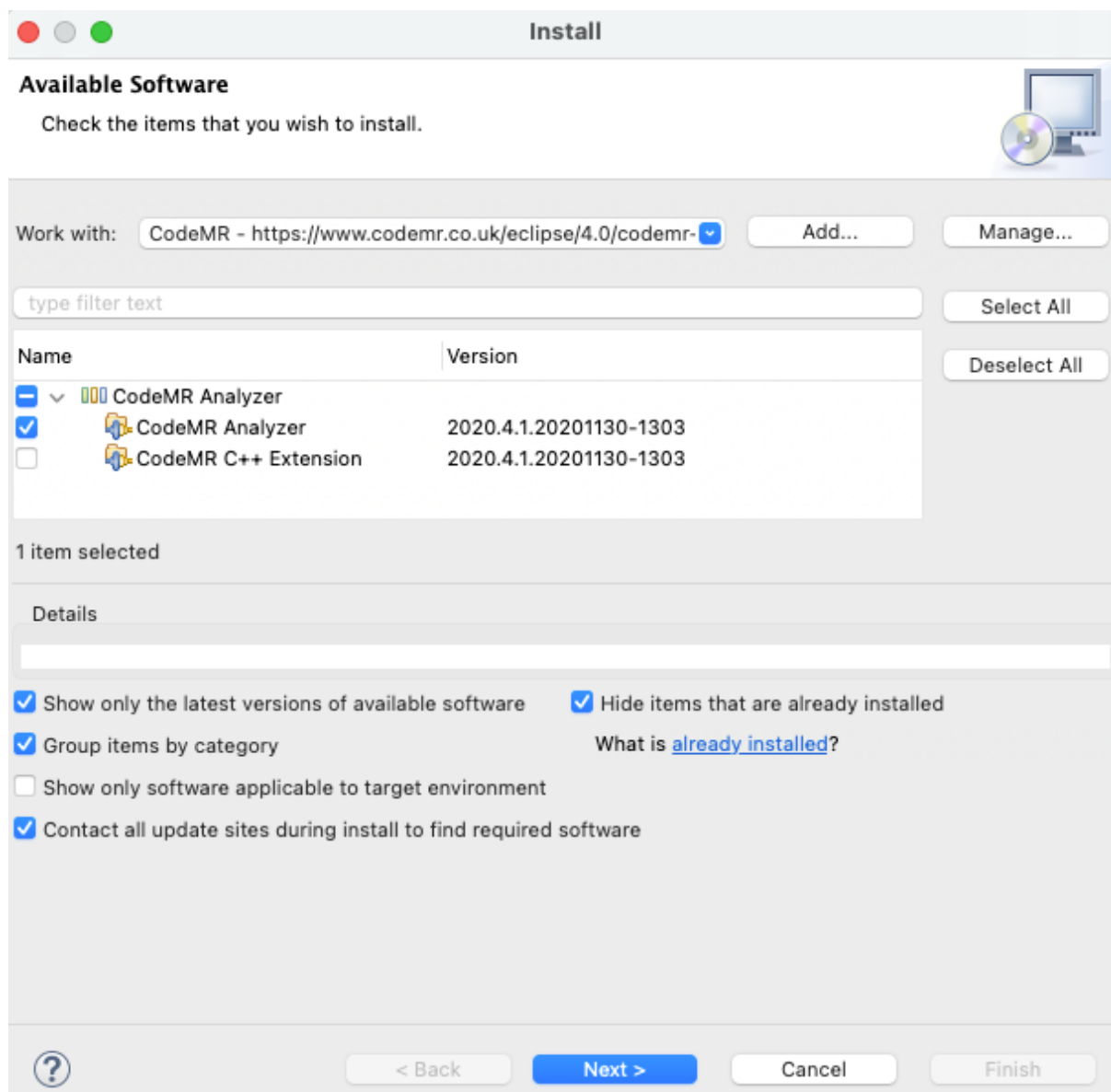


Figure 5. Review Installation Details

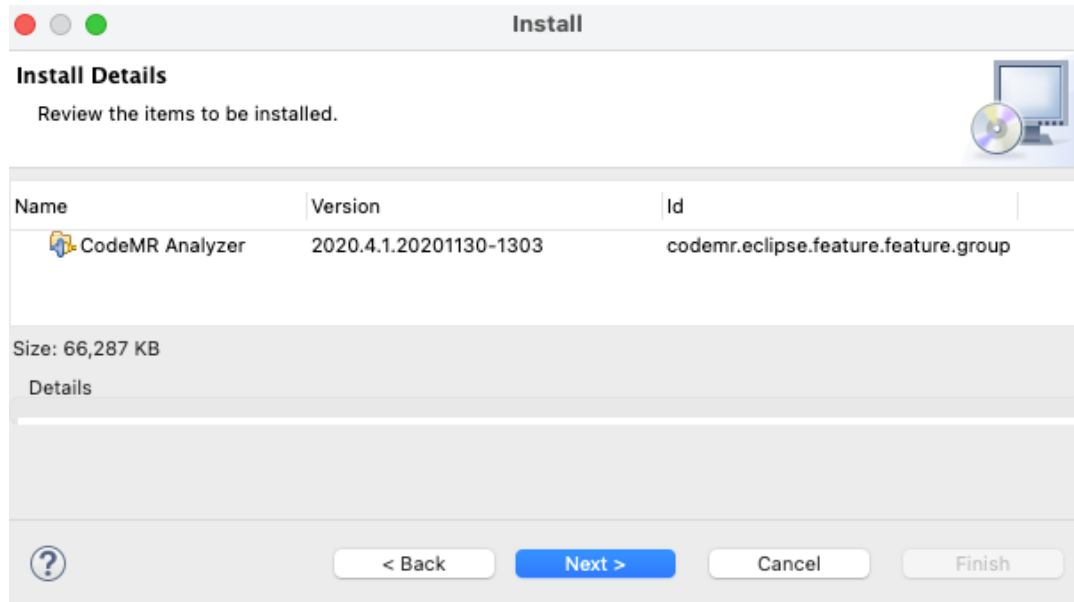


Figure 6. Accept EULA

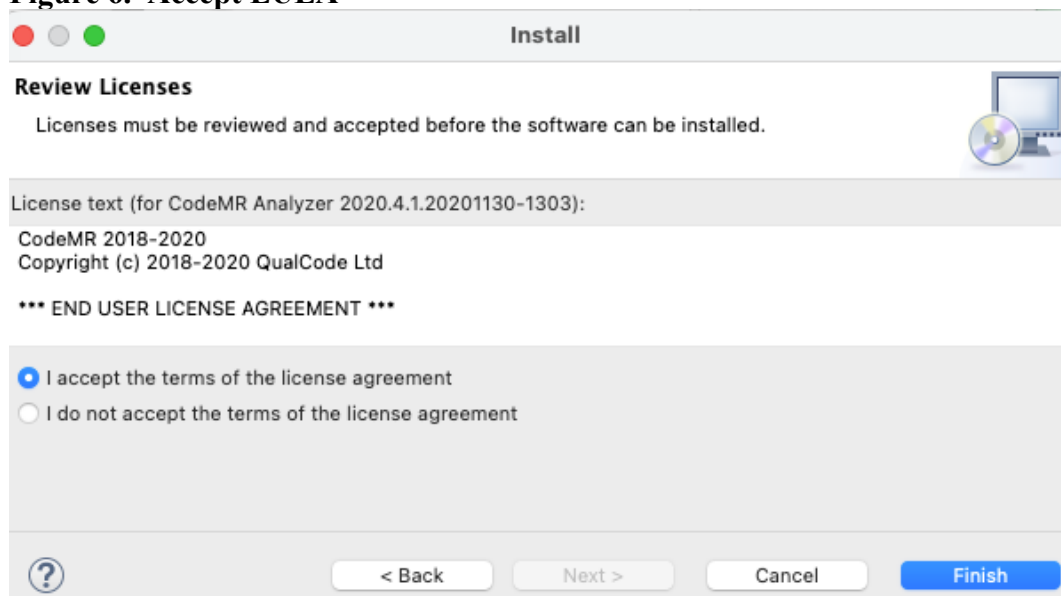


Figure 7. Trust Authorities

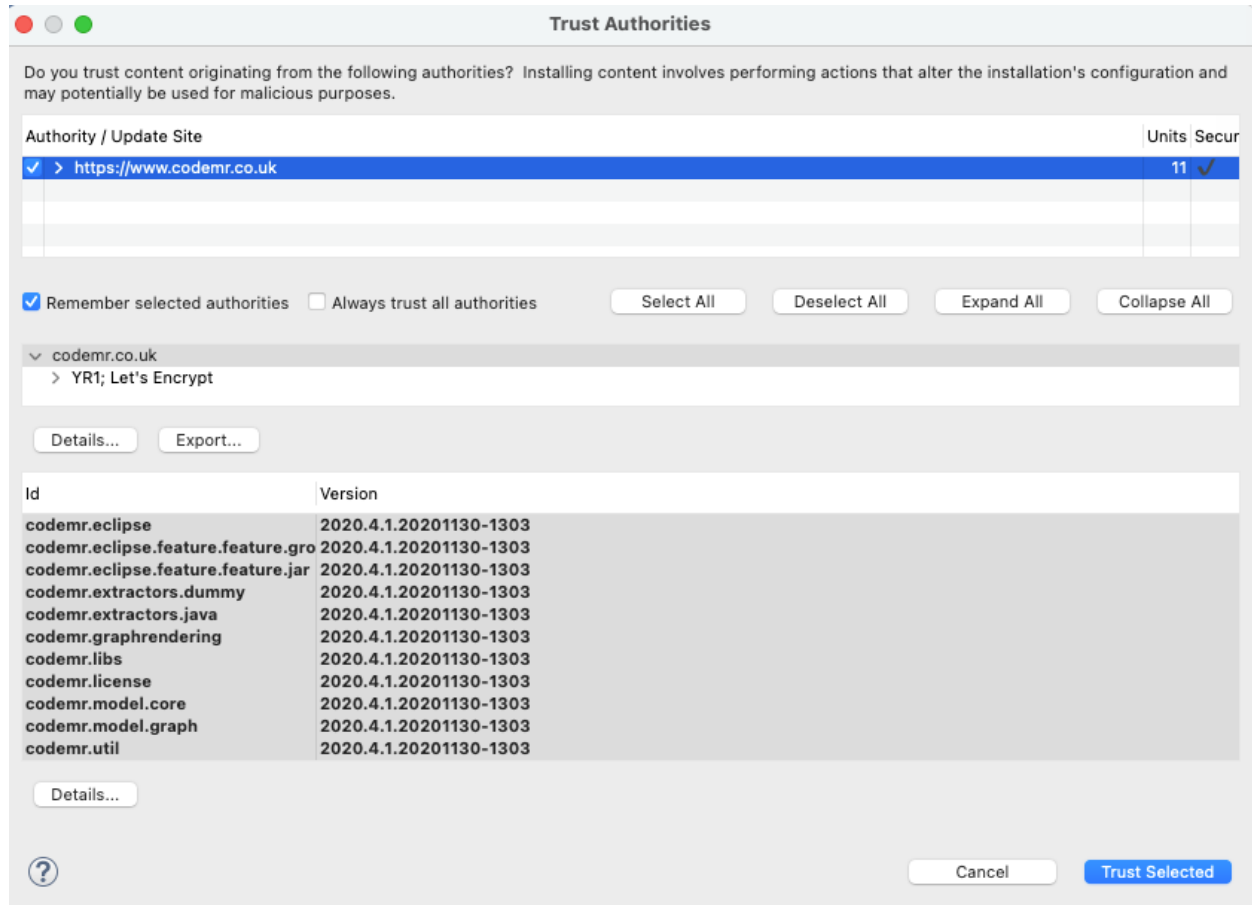


Figure 8. Eclipse Installation Progress



Figure 9. Trust Unsigned Artifacts

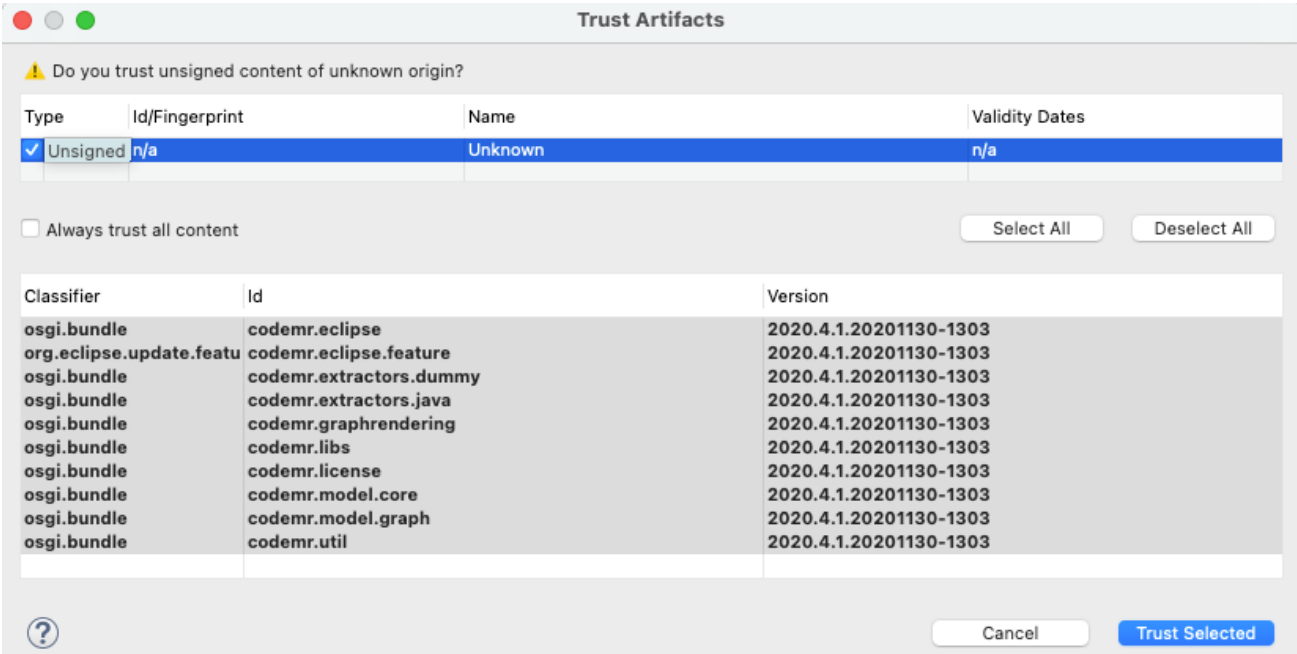


Figure 10. Restart Eclipse

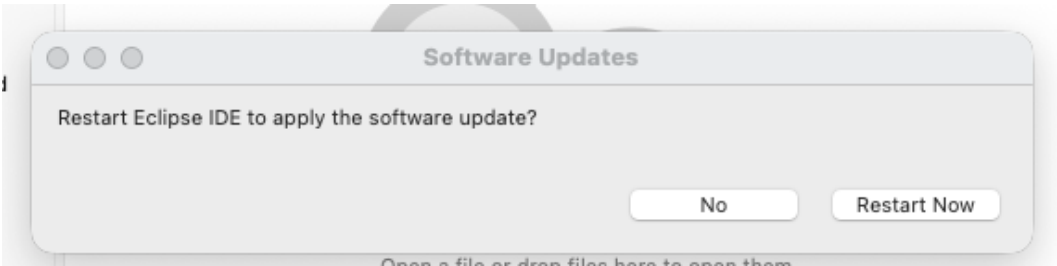


Figure 11. Verify CodeMR Installation

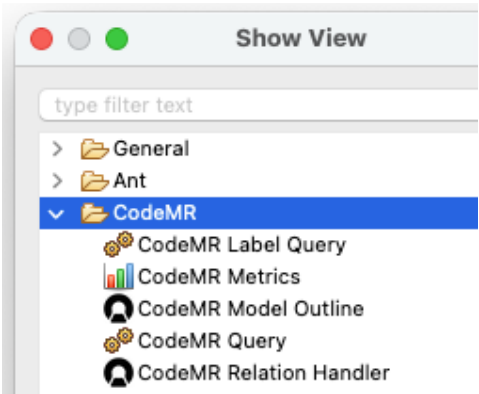


Figure 12. Open the CodeMR Perspective

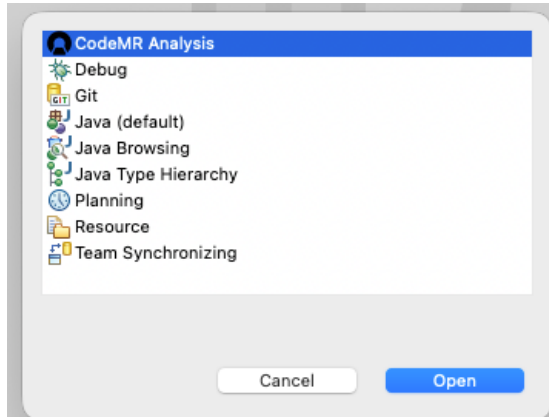


Figure 13. Run CodeMR Analysis

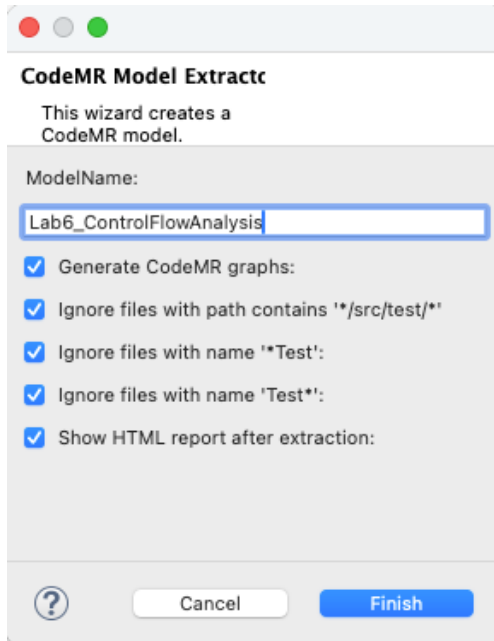


Figure 14. CodeMR Analysis Error



Figure 15. CodeMR Analysis Error When Using Trial License from Website

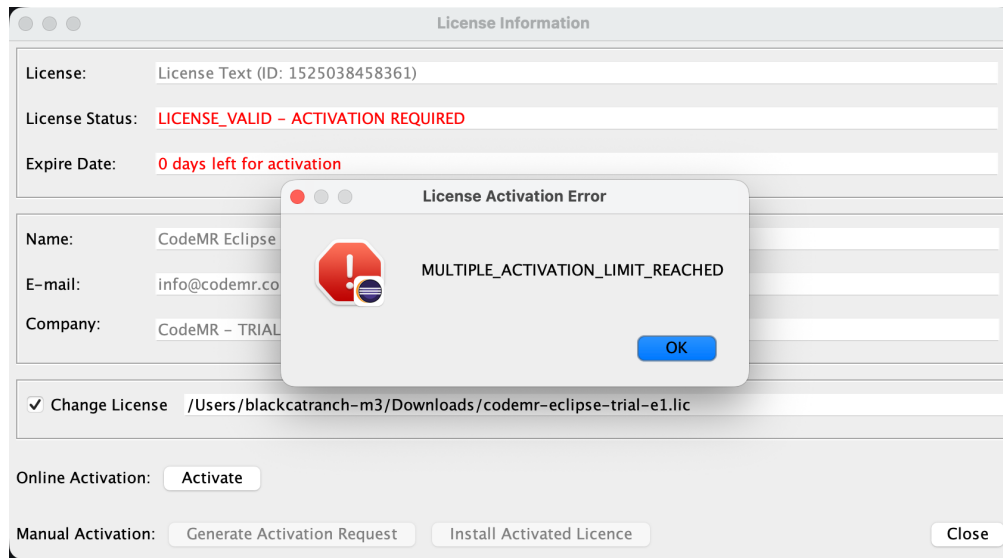


Figure 16. CodeMR Email Exchange and New Trial License

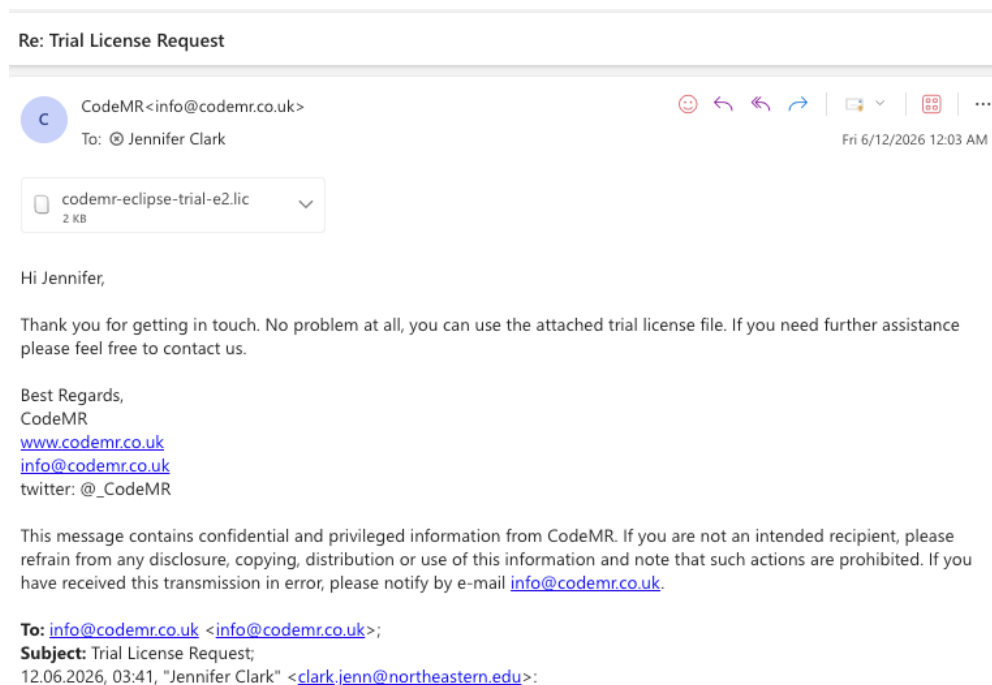


Figure 17. CodeMR New Trial License Installation

The 'License Information' dialog box displays the following details:

- License:** License Text (ID: 1578563675298)
- License Status:** LICENSE_VALID – ACTIVATION COMPLETED
- Expire Date:** Fri Jun 26 07:12:27 PDT 2026
- Name:** CodeMR Eclipse Plugin
- E-mail:** info@codemr.co.uk
- Company:** CodeMR

Below the fields, there is a checked checkbox labeled 'Change License' followed by the file path: /Users/blackcatranch-m3/Downloads/codemr-eclipse-trial-e2.lic.

At the bottom, there are two sections for activation:

- Online Activation:** An 'Activate' button.
- Manual Activation:** 'Generate Activation Request' and 'Install Activated Licence' buttons.

A 'Close' button is located in the bottom right corner.

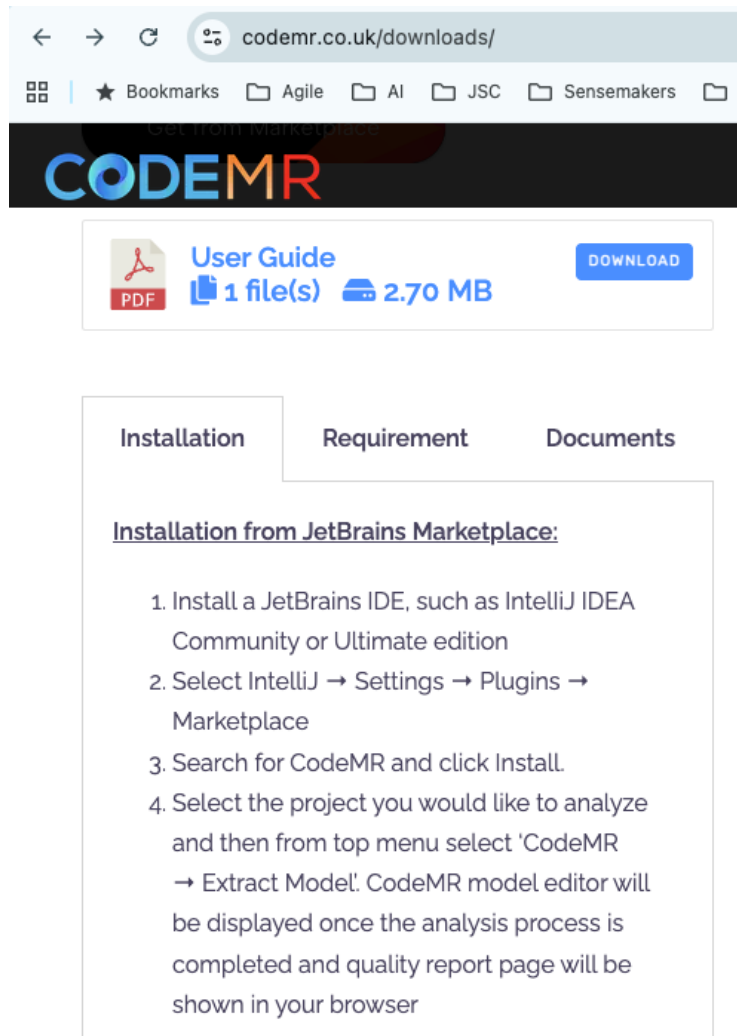
Figure 18. CodeMR Trial License Activation Completed

The 'License Activation OK' dialog box is displayed over the 'License Information' dialog box. It features the CodeMR logo (a stylized 'C' with horizontal lines) and the text 'ACTIVATION_COMPLETED'. An 'OK' button is located at the bottom right of the dialog box.

The background 'License Information' dialog box shows the same information as in Figure 17, including the license ID, status, expire date, name, email, and company.

APPENDIX 2. JetBrains IntelliJ IDEA Installation

Figure 19. Installation instructions for IntelliJ IDEA



The screenshot shows a web browser at the URL `codemr.co.uk/downloads/`. The browser's bookmark bar includes 'Bookmarks', 'Agile', 'AI', 'JSC', and 'Sensemakers'. The CodeMR logo is prominently displayed. Below the logo, a download section for the 'User Guide' is shown, indicating it is a PDF file (1 file(s), 2.70 MB) with a 'DOWNLOAD' button. Below this, a tabbed interface has 'Installation' selected. The 'Installation' tab contains the heading 'Installation from JetBrains Marketplace:' followed by a four-step numbered list:

1. Install a JetBrains IDE, such as IntelliJ IDEA Community or Ultimate edition
2. Select IntelliJ → Settings → Plugins → Marketplace
3. Search for CodeMR and click Install.
4. Select the project you would like to analyze and then from top menu select 'CodeMR → Extract Model'. CodeMR model editor will be displayed once the analysis process is completed and quality report page will be shown in your browser

Figure 20. Find & Install Code MR

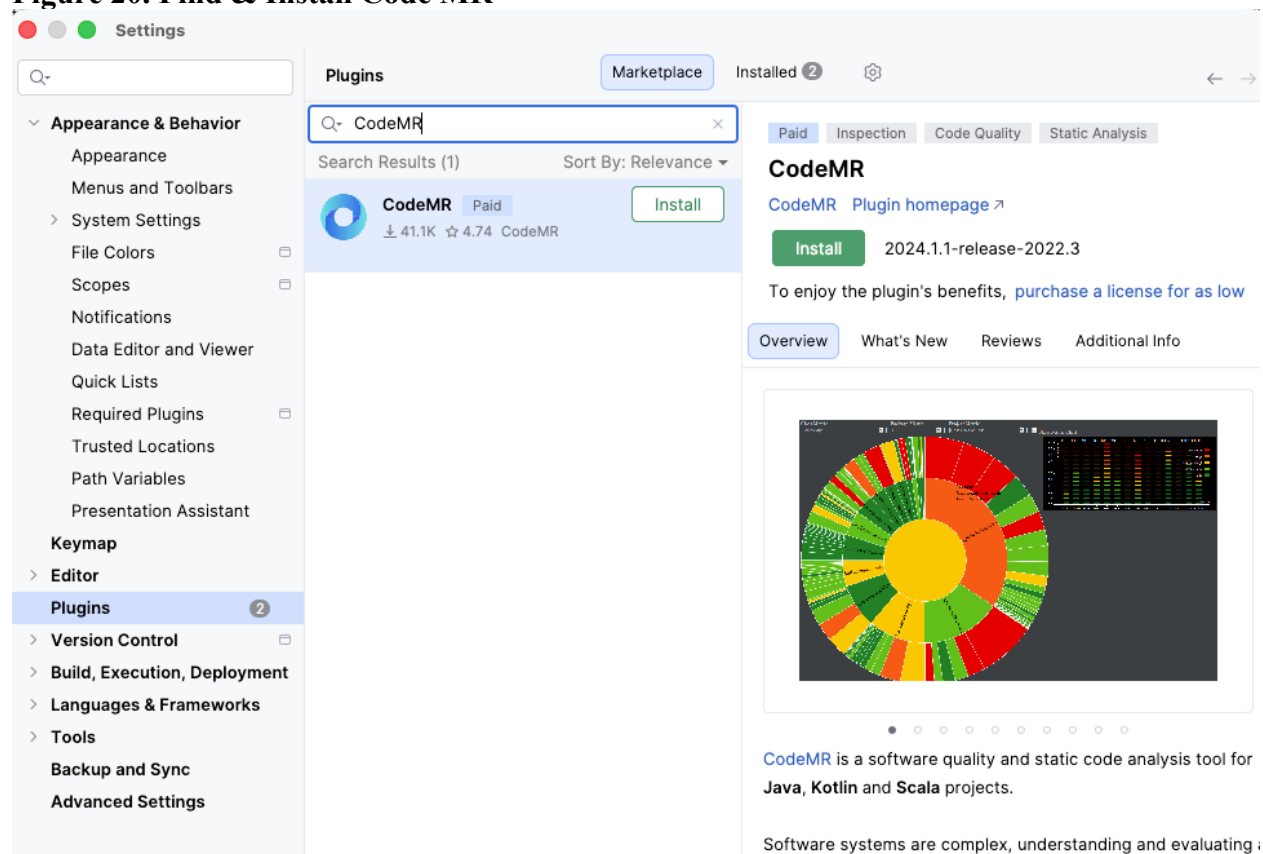


Figure 21. Set up Trial Account

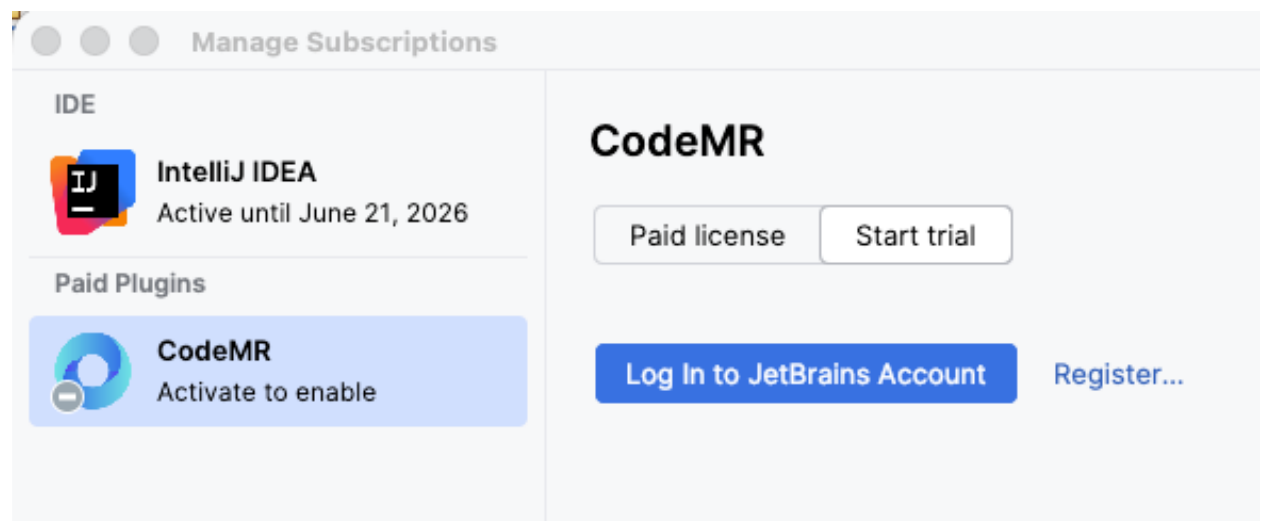


Figure 22. Apply for a JetBrains Student or Teacher Pack

<https://www.jetbrains.com/shop/eform/v2/students>

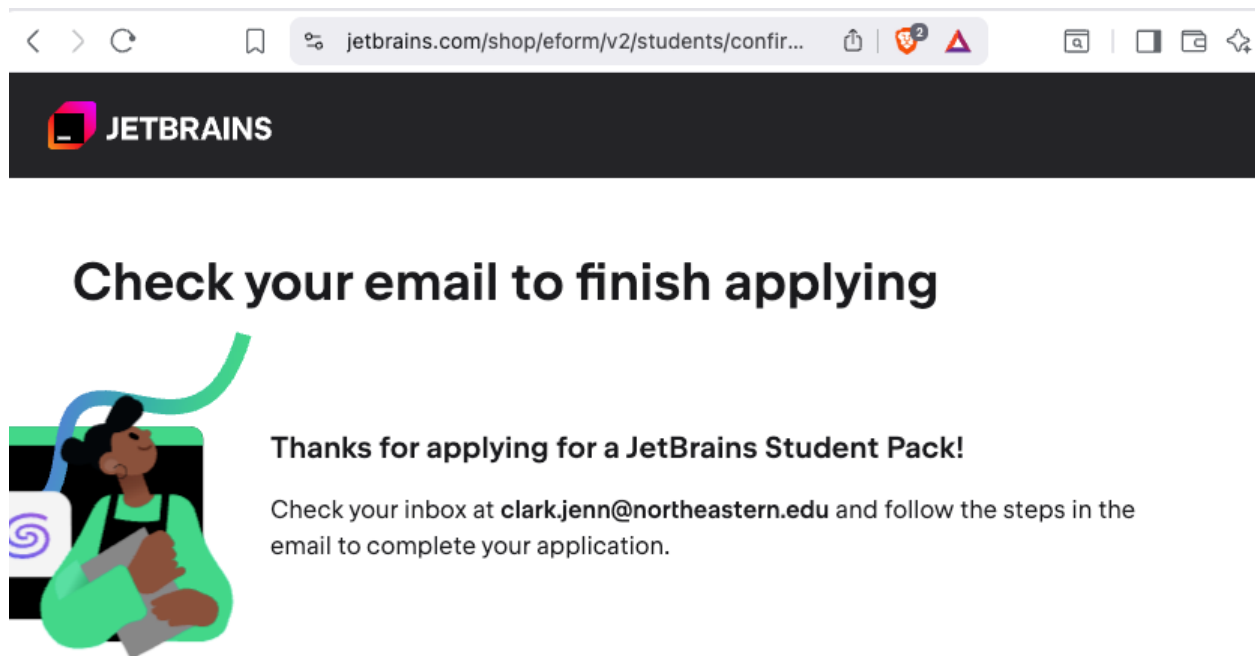


Figure 23. Apply for a JetBrains Student or Teacher Pack (continued)

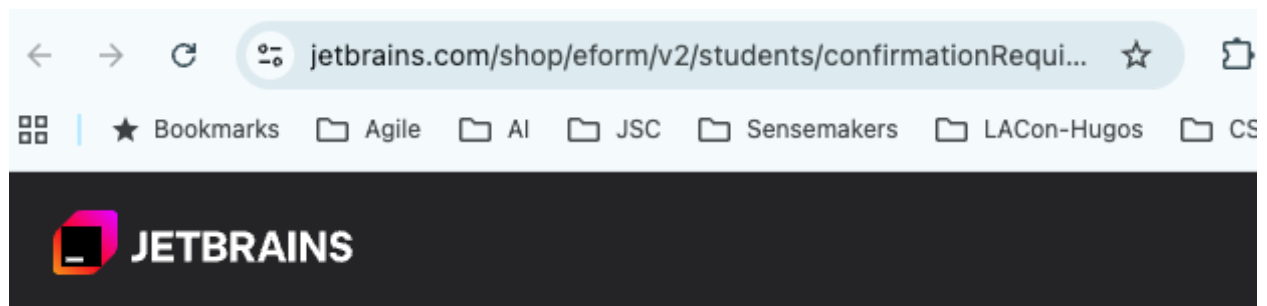


Figure 24. Authorize Git Access for JetBrains Account

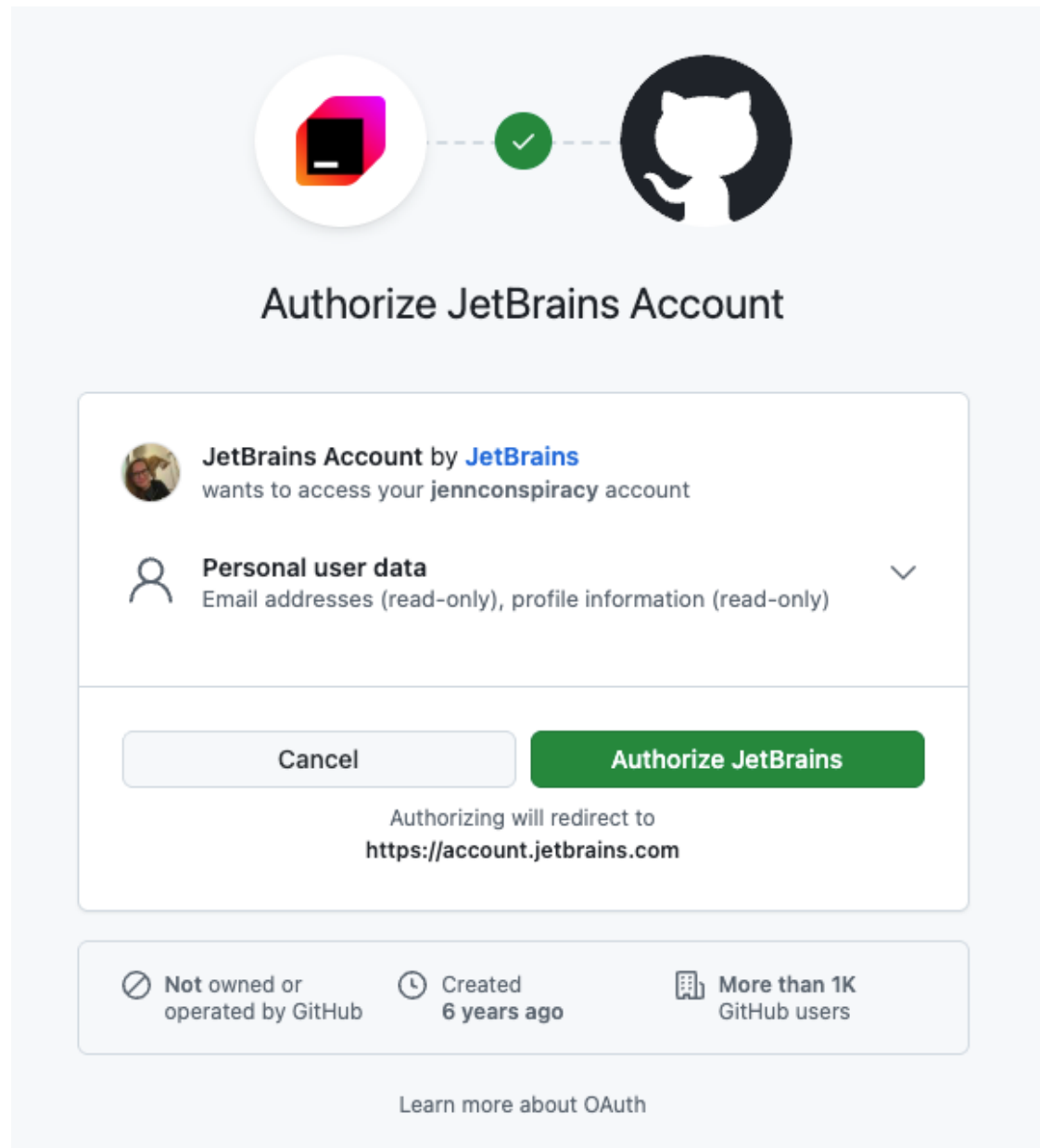
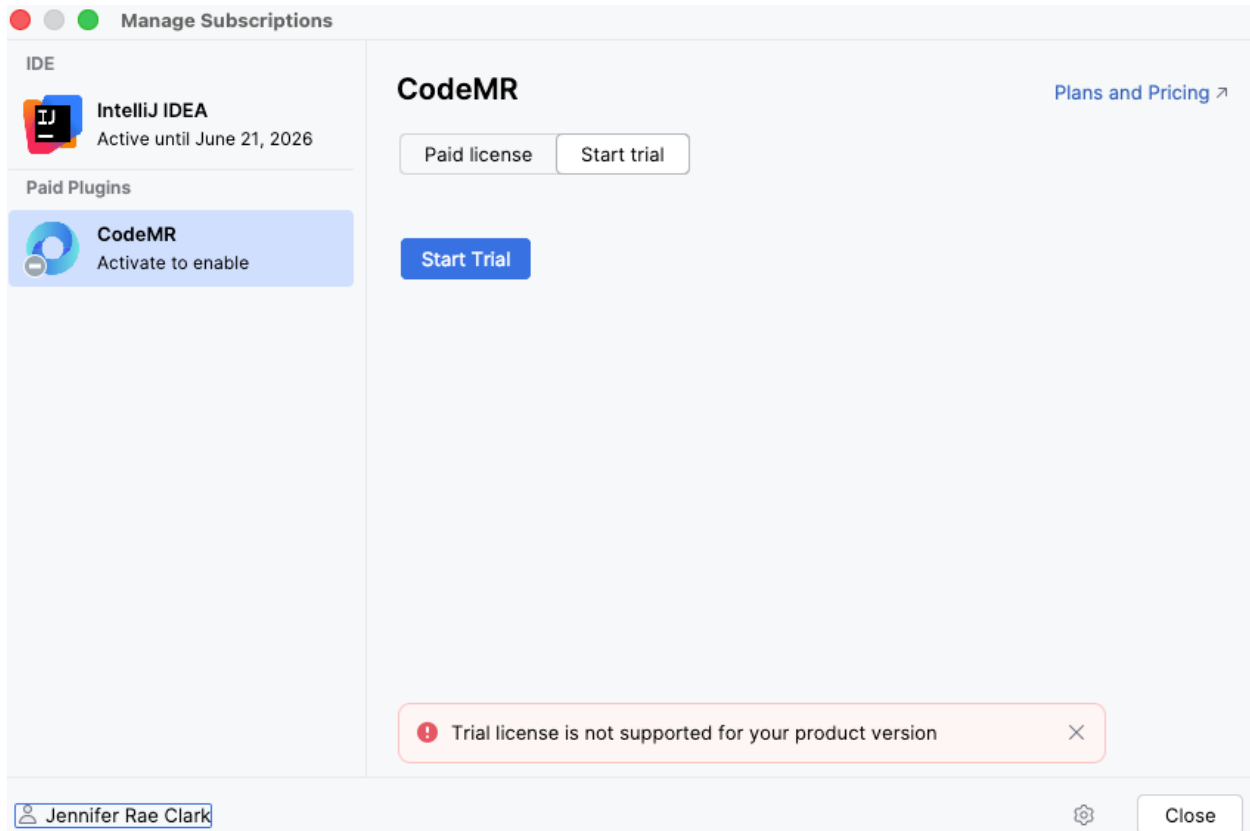


Figure 25. Install CodeMR on Jetbains



NOTE: there's a student rate for \$15/month <https://plugins.jetbrains.com/plugin/10811-codemr/pricing?fromIDE&lang=en-us#personal>

APPENDIX 3: CodeMR Report Comparative Summaries

Table 1: Project Metrics Summary

	Info5100- University- Example	Assignment02	RangePricing	Midterm	Final
LOC	581	1826	2430	10141	9317
CLASSES	39	14	62	74	89
PACKAGES	12	2	18	20	9
EXTERNAL PACKAGES	4	26	30	33	56
EXTERNAL CLASSES	12	168	190	259	588
PROB CLASSES	0	0	0	3	23
HI PROB CLASSES	0	0	0	0	0
APPENDIX FIGURES	4	5	6	7	8

Table 2: Project “C” Metrics Summary

	Info5100- University- Example	Assignment02	RangePricing	Midterm	Final
C3	76.4% low	94.2% med-hi	71.6% med-hi	80% med-hi 11.9% hi/v-hi	69.3% hi
Complexity	100% low	92.1% med-hi	58.6% med-hi	77.2 med-hi	69.3% hi
Coupling	83% low	100% low/med-low	74.4% low/med-low	85.2% low/med-low	65.9% low/med-low
Cohesion	86.9% low	97.8% low/med-low	82.5% low/med-low	64% low/med-low	55.4% low/med-low

Figure 26. Analysis & Distribution of Quality Attributes

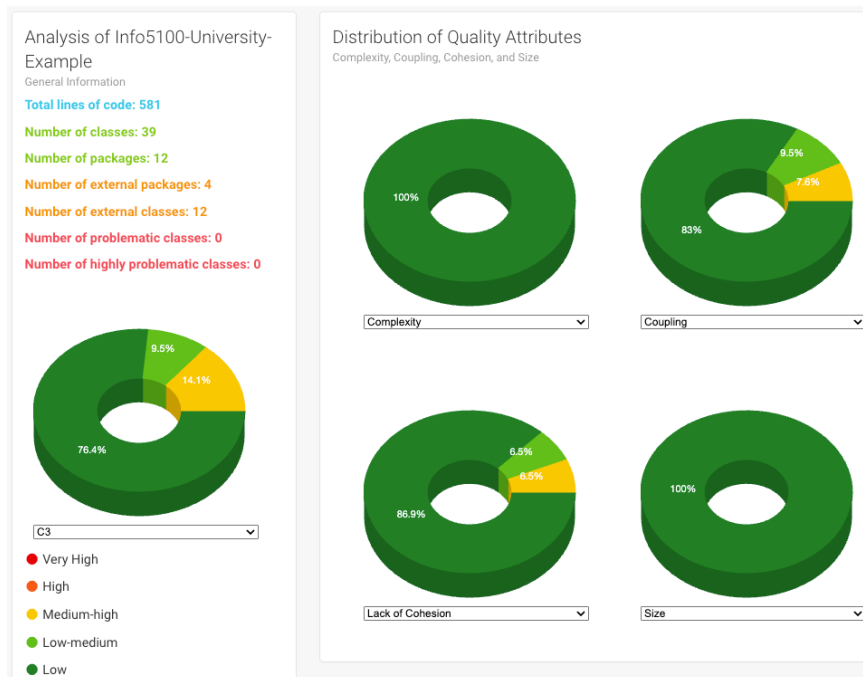


Figure 27. Metric Values in Sunburst Chart

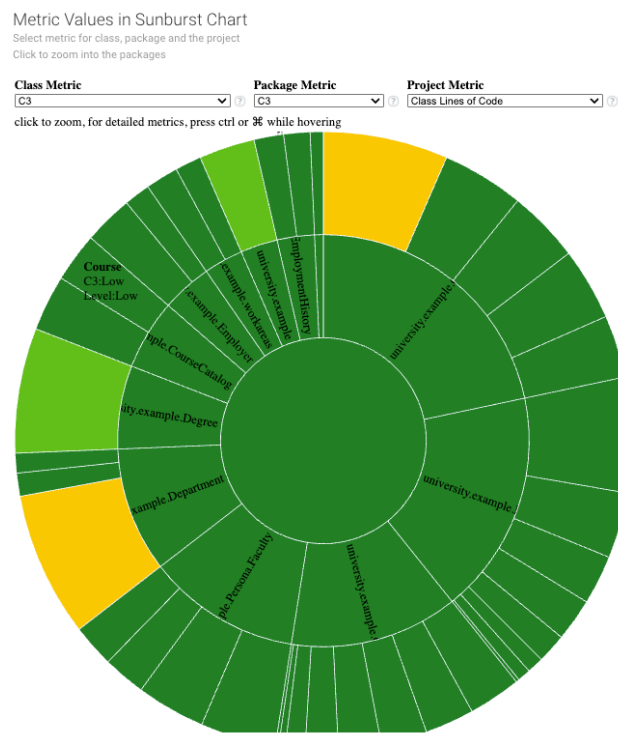


Figure 28. Metric Values in Package

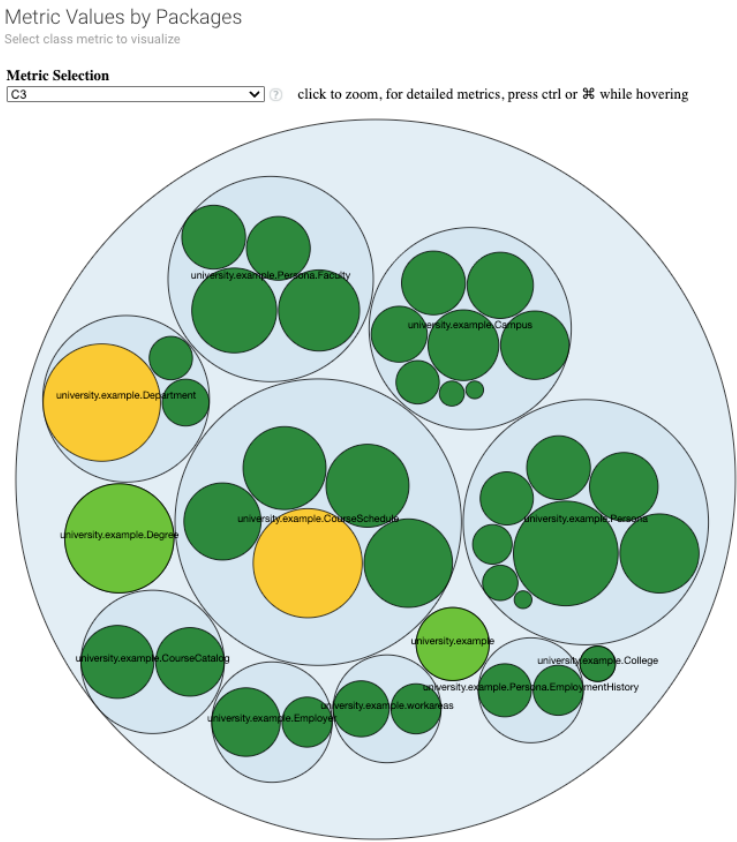


Figure 29. Metric Values in Treemap Chart

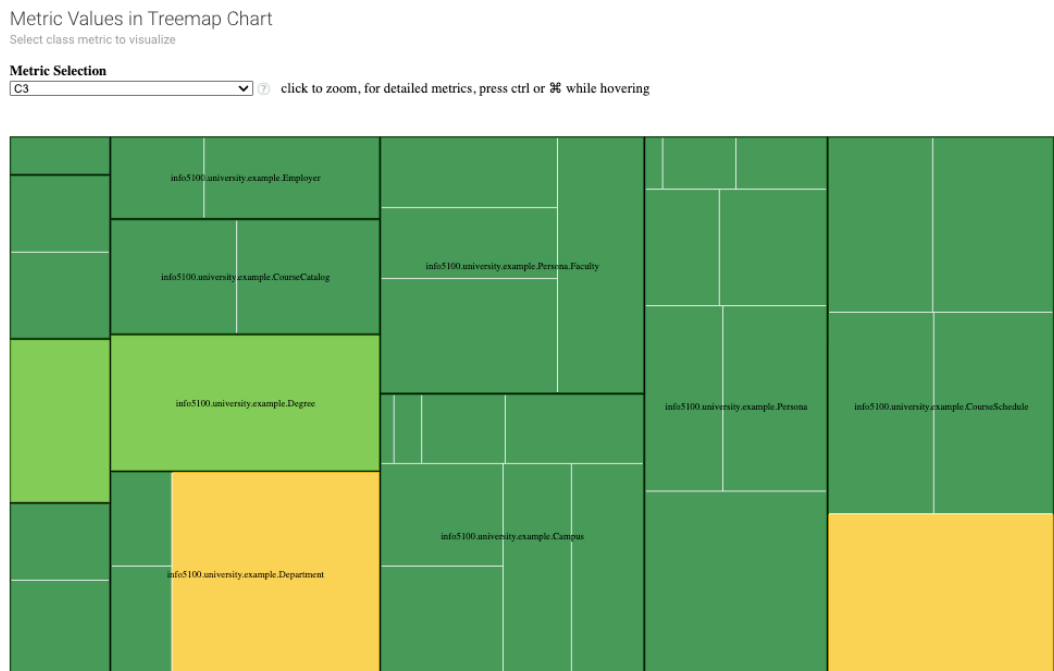
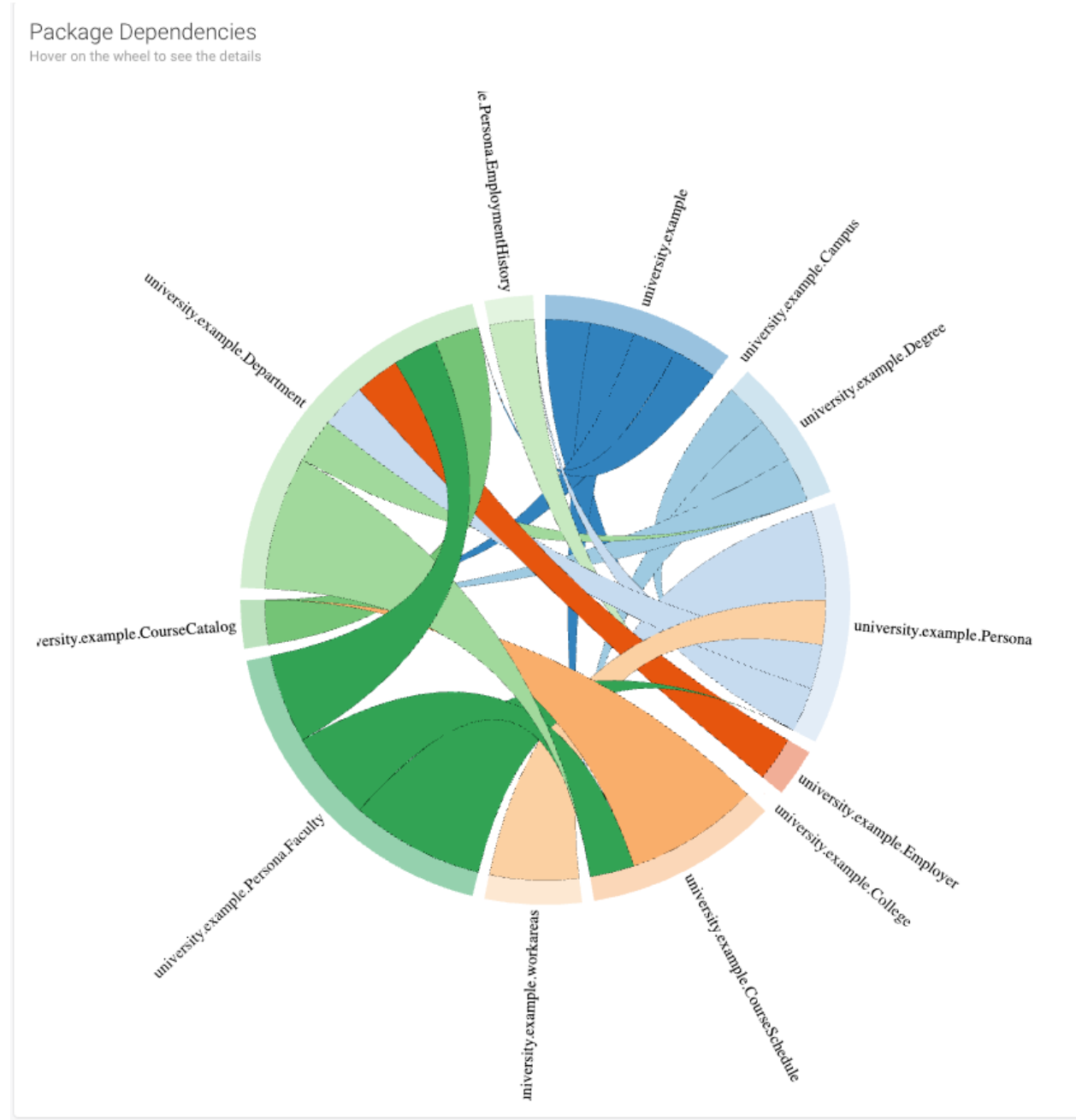


Figure 30. Package Dependencies



APPENDIX 5: Assignment02

Figure 31. Analysis & Distribution of Quality Attributes

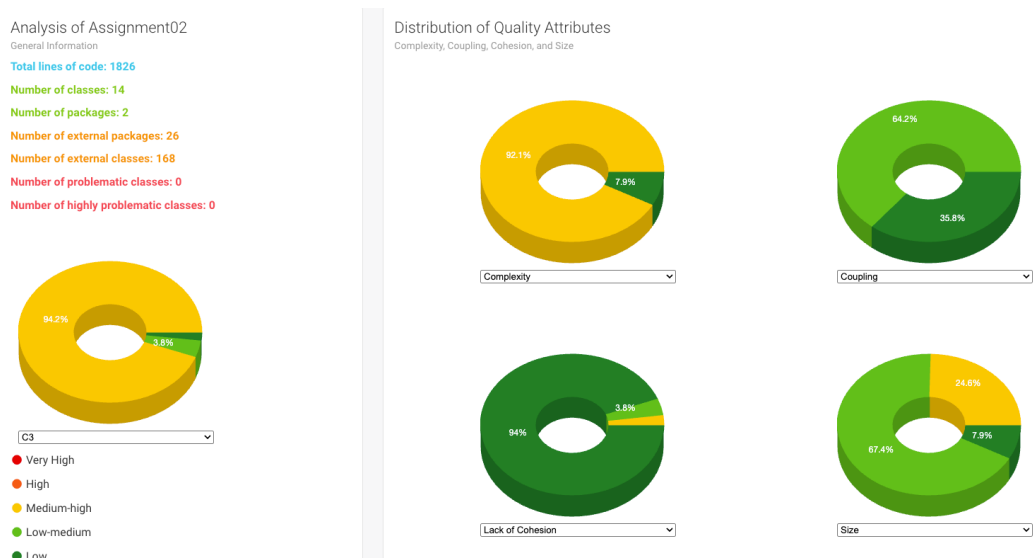


Figure 33. Metric Values in Package

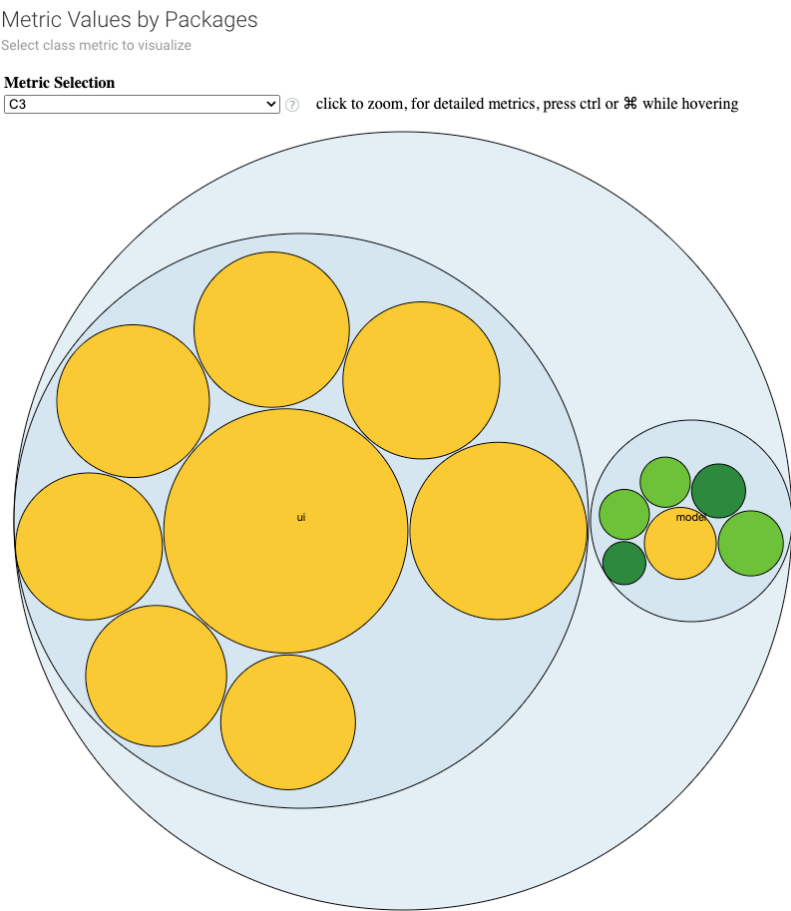


Figure 34. Metric Values in Treemap Chart

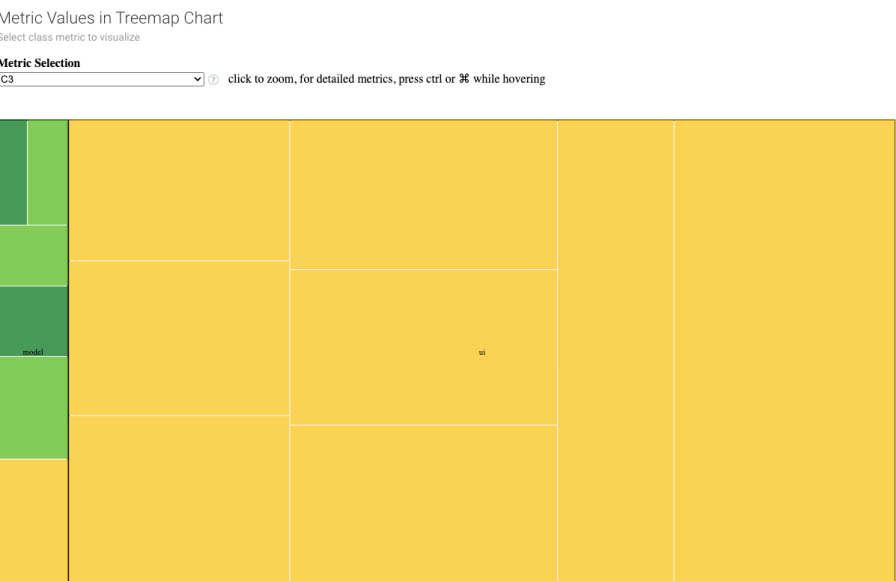


Figure 35. Package Dependencies

Package Dependencies
Hover on the wheel to see the details

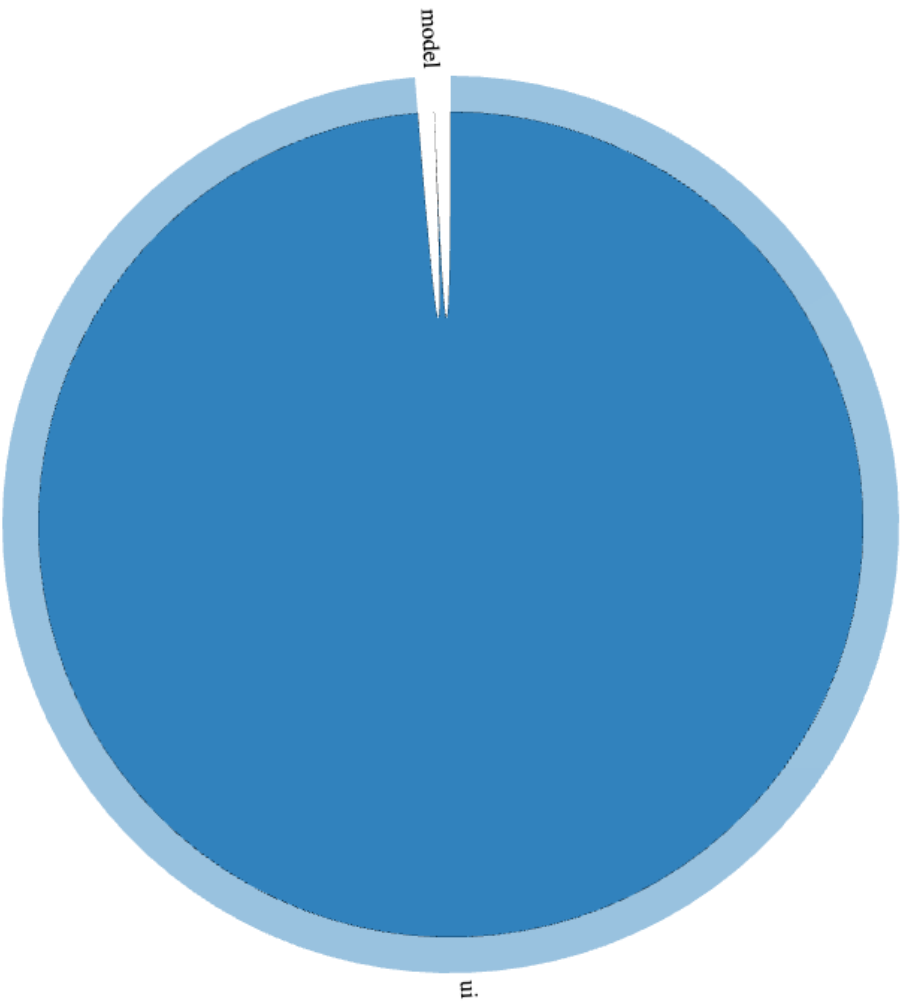


Figure 36. Analysis & Distribution of Quality Attributes



Figure 37. Metric Values in Sunburst Chart

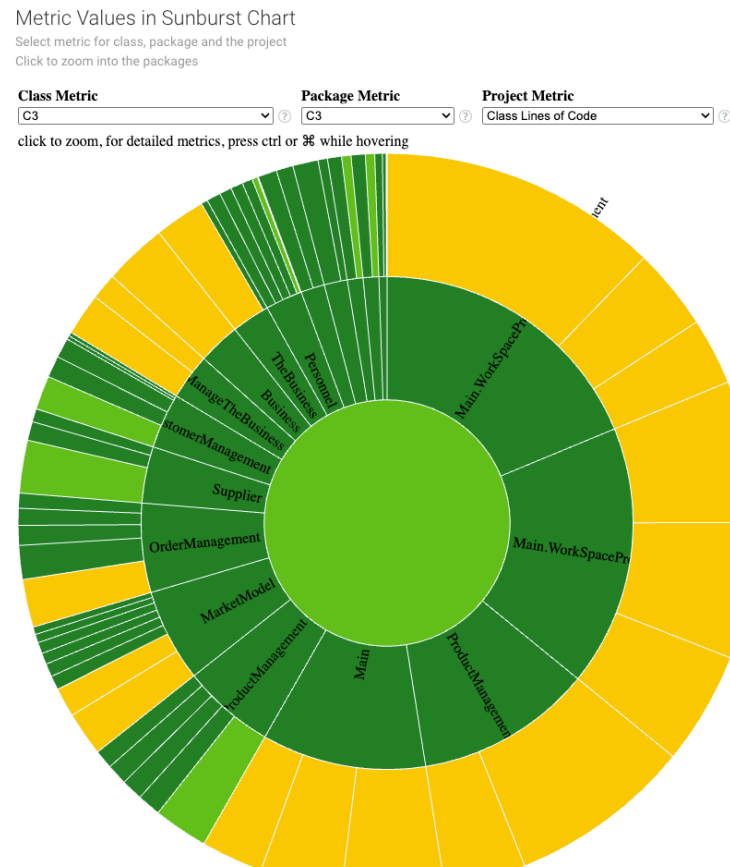


Figure 38. Metric Values in Package

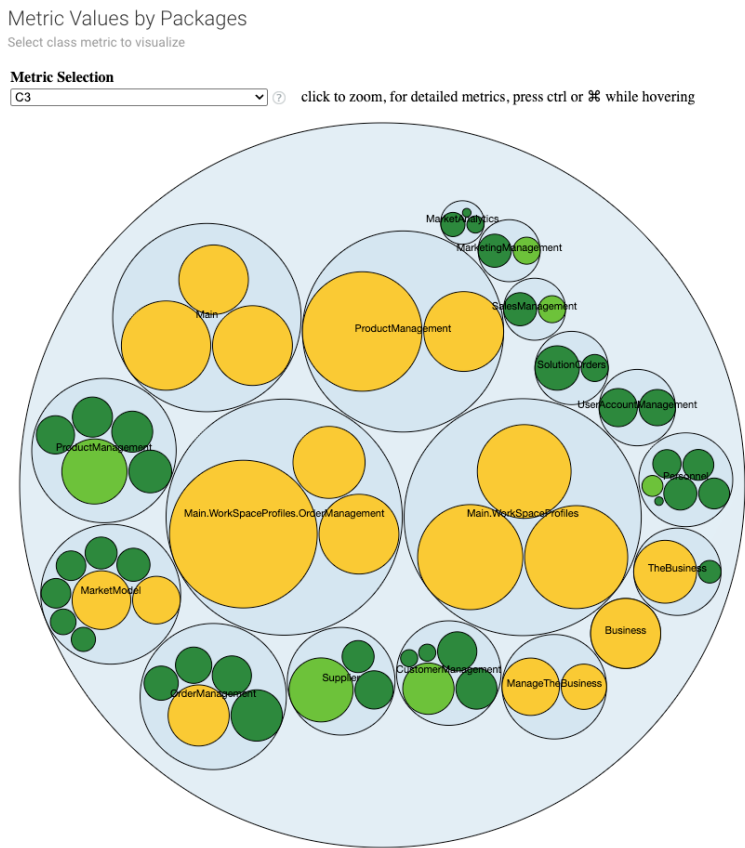


Figure 39. Metric Values in Treemap Chart

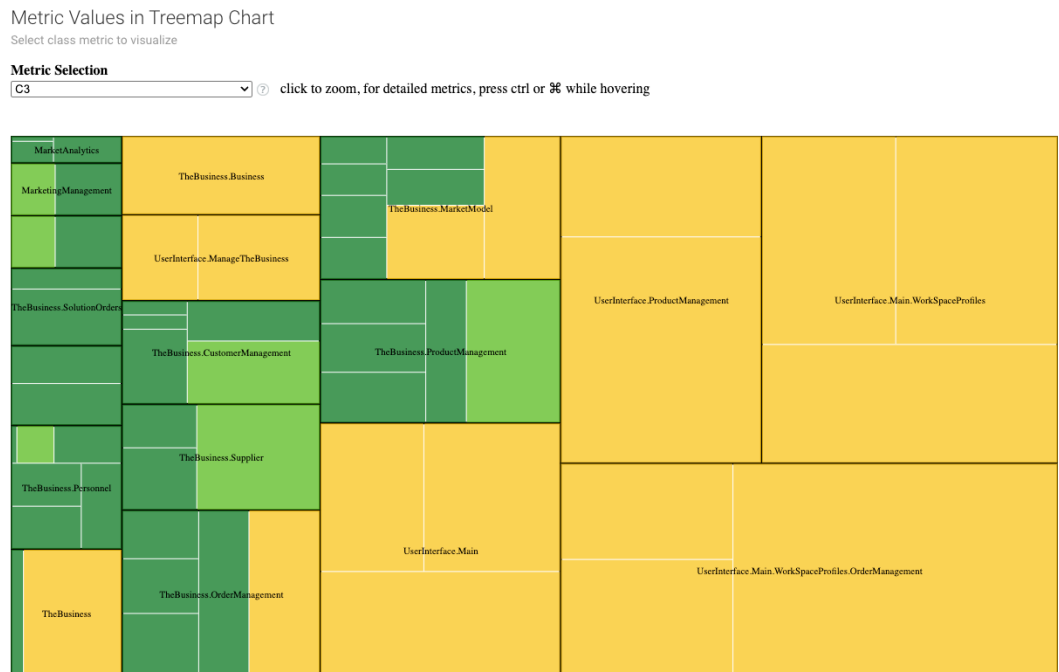
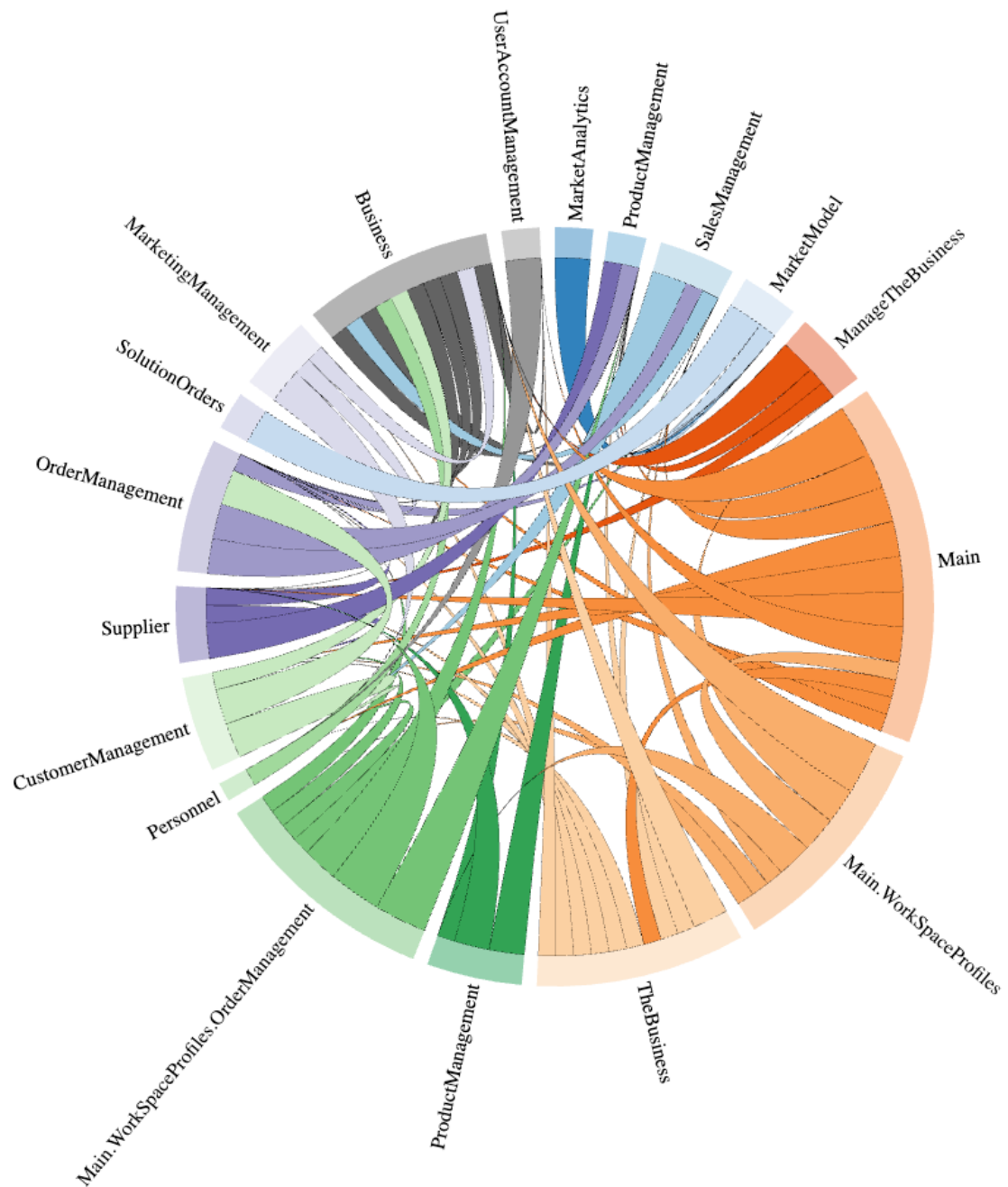


Figure 40. Package Dependencies

Package Dependencies

Hover on the wheel to see the details

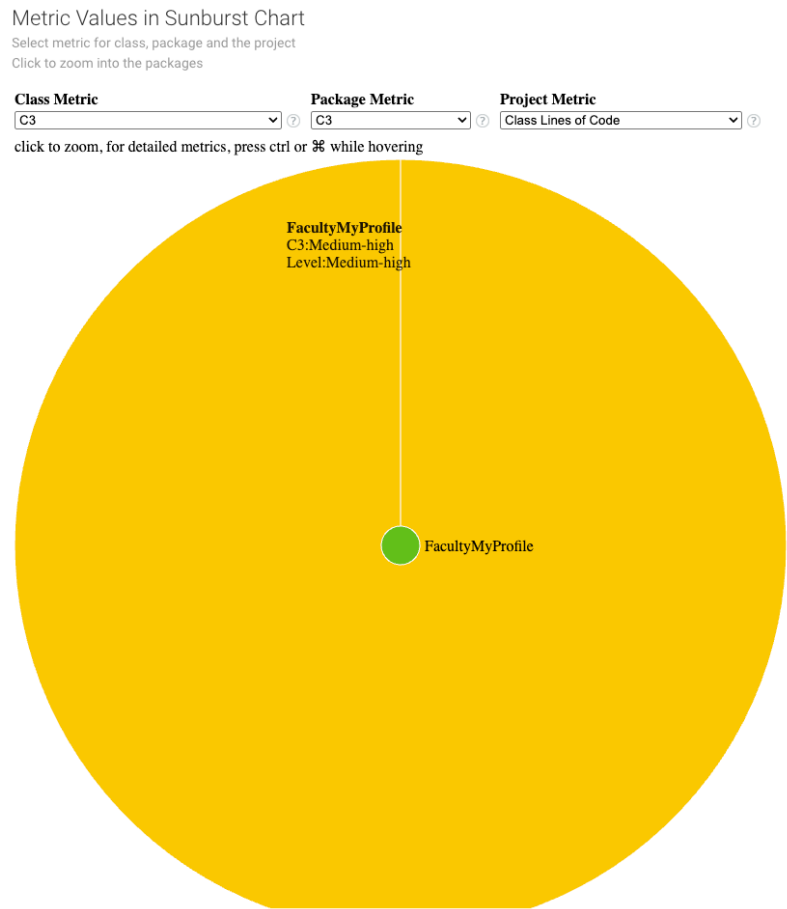


APPENDIX 7: Midterm_Project_JavaHuskies

Figure 41. Analysis & Distribution of Quality Attributes



Figure 42. Metric Values in Sunburst Chart



Metric Values by Packages

Metric Selection

C3 click to zoom, for detailed metrics, press ctrl or ⌘ while hovering



Metric Selection

C3 click to zoom, for detailed metrics, press ctrl or ⌘ while hovering

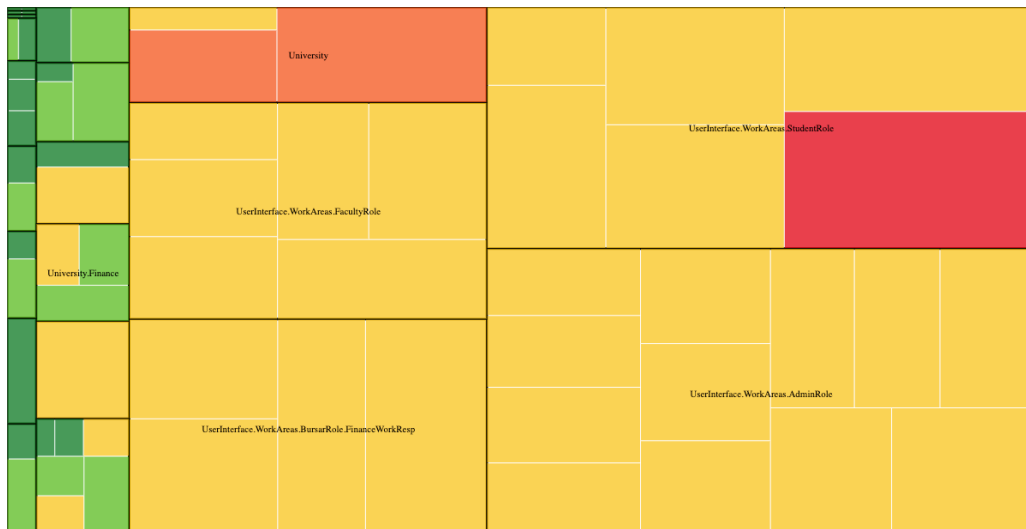
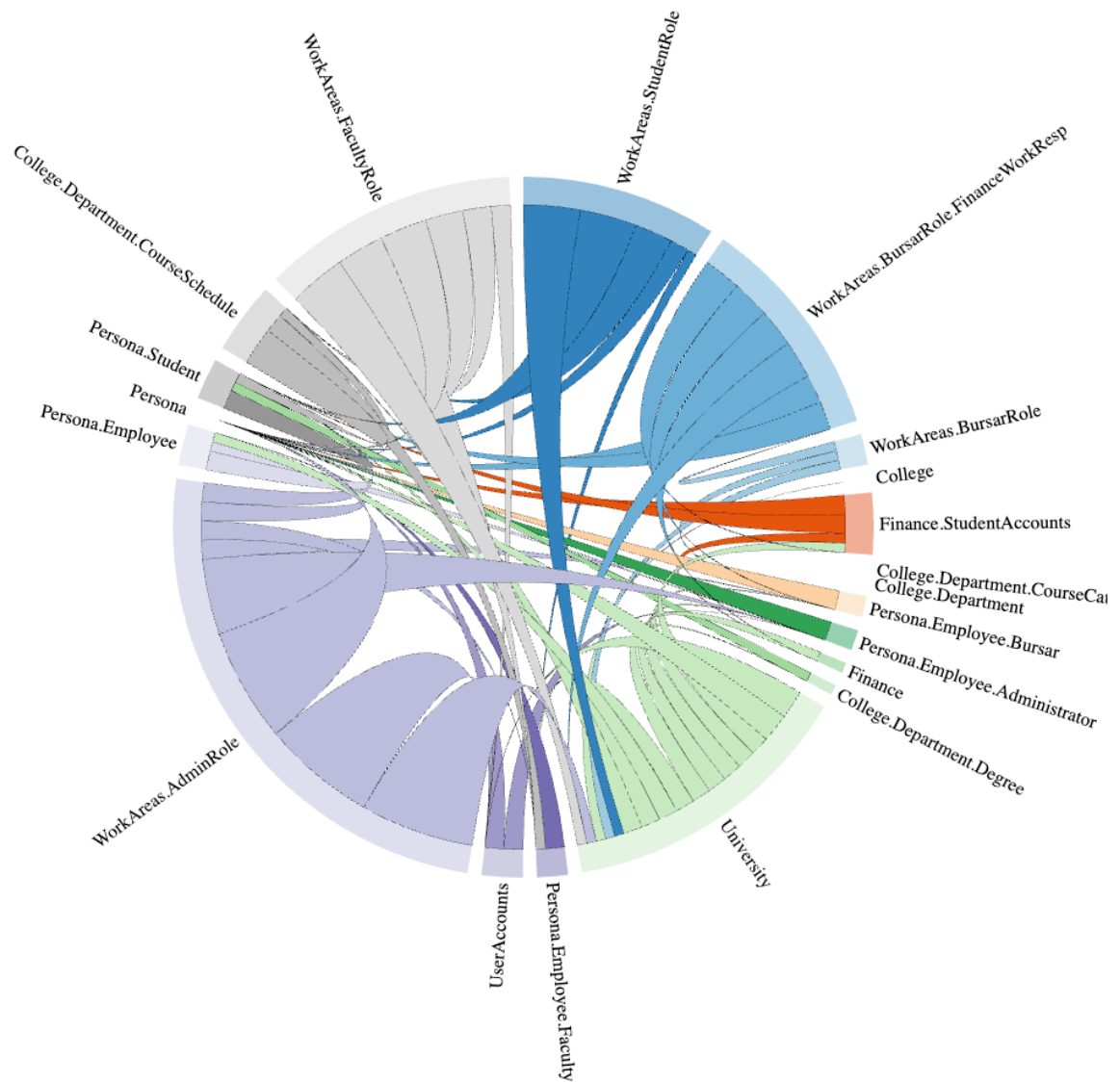


Figure 45. Package Dependencies

Package Dependencies

Hover on the wheel to see the details



APPENDIX 8: FinalProject_JavaHuskies

Figure 46. Analysis & Distribution of Quality Attributes



Figure 47. Metric Values in Sunburst Chart

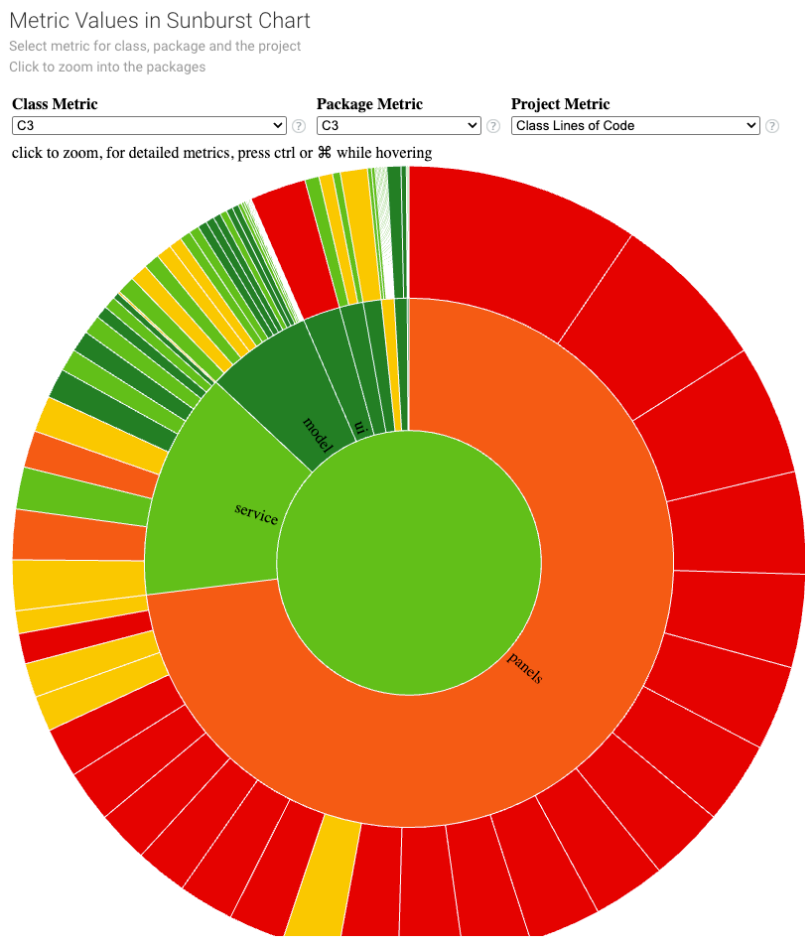


Figure 48. Metric Values in Package

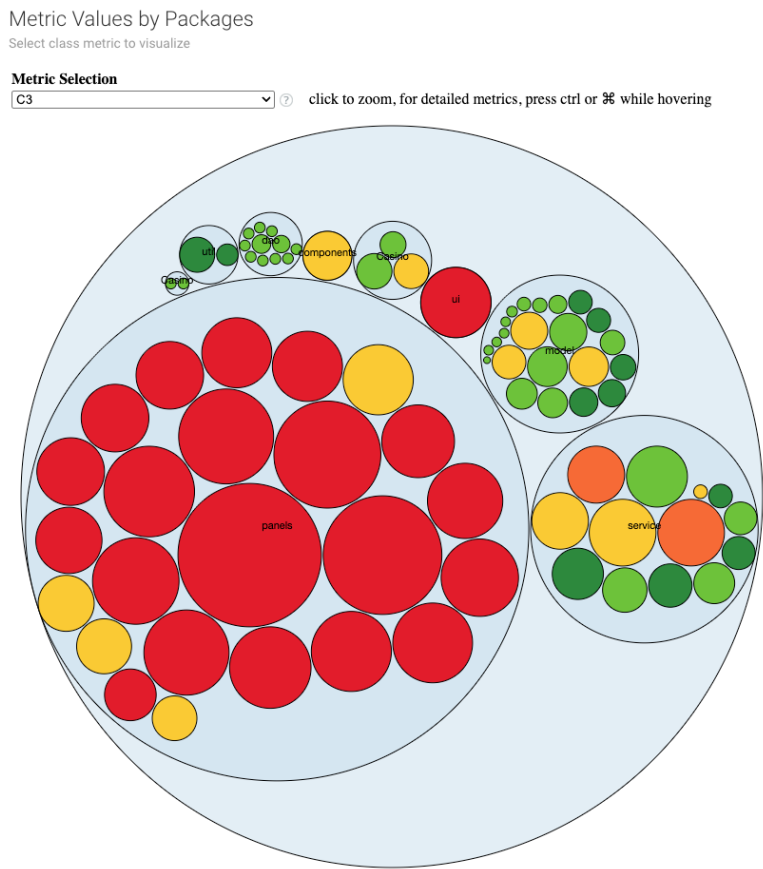


Figure 49. Metric Values in Treemap Chart

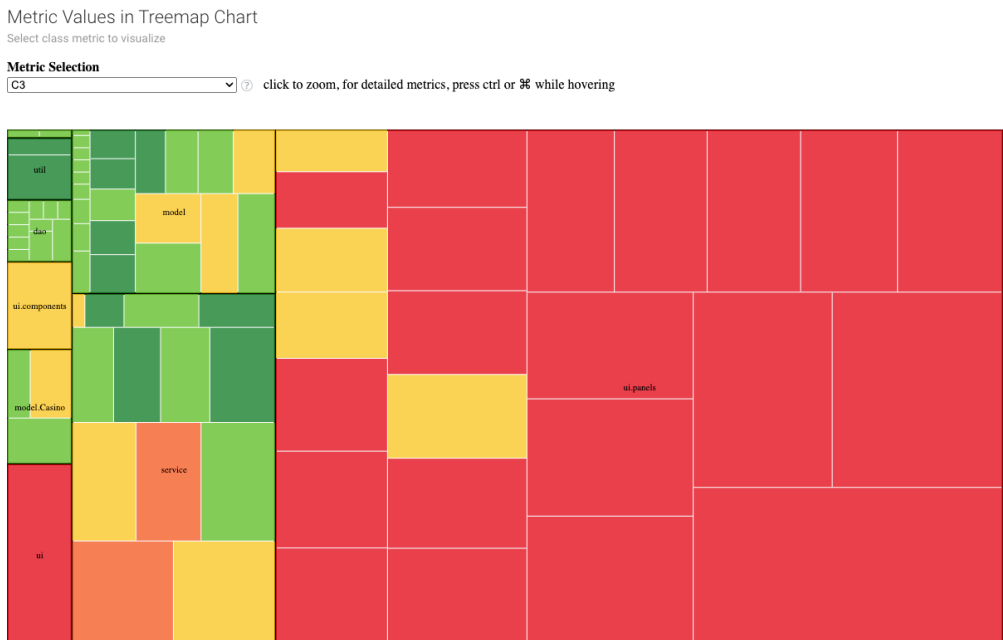
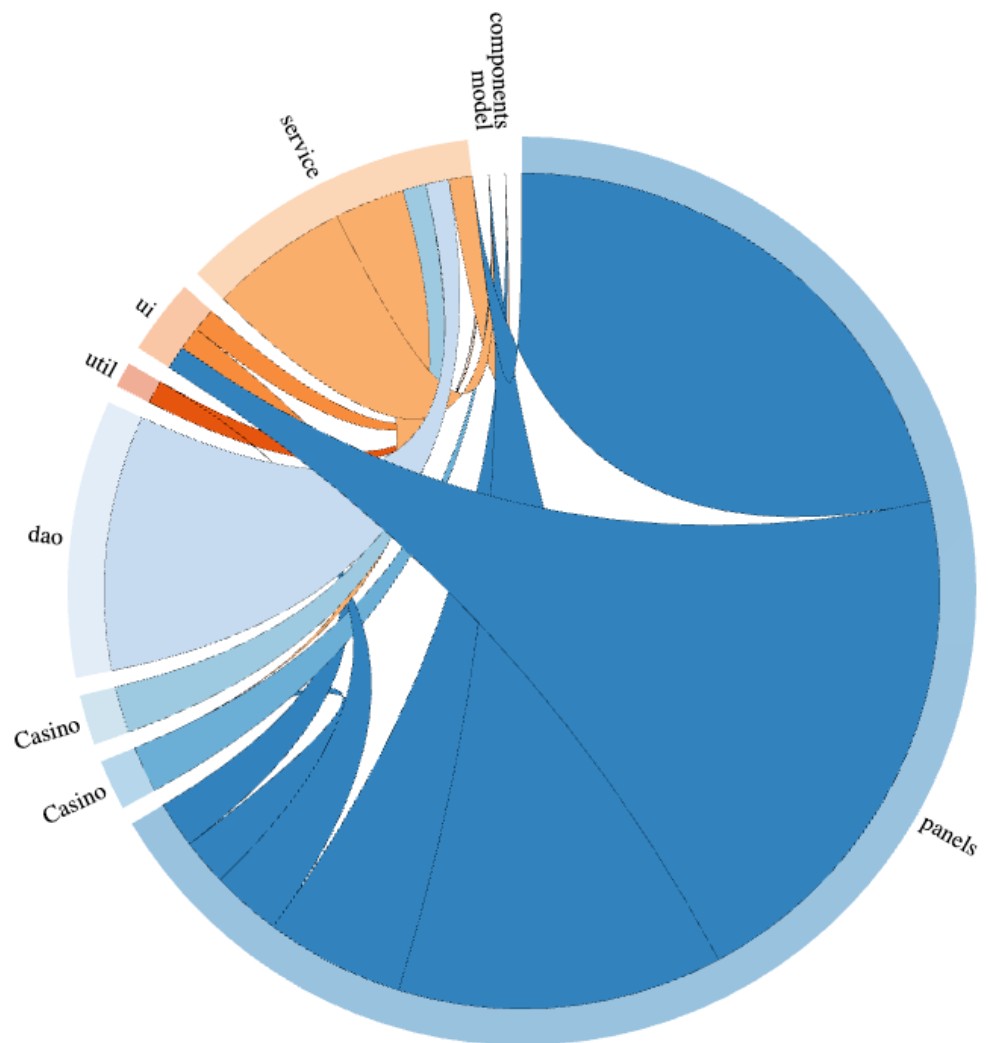


Figure 50. Package Dependencies

Package Dependencies

Hover on the wheel to see the details



APPENDIX 9: Other CodeMR Artifacts

Figure 52. Detailed Metric List (partial capture)

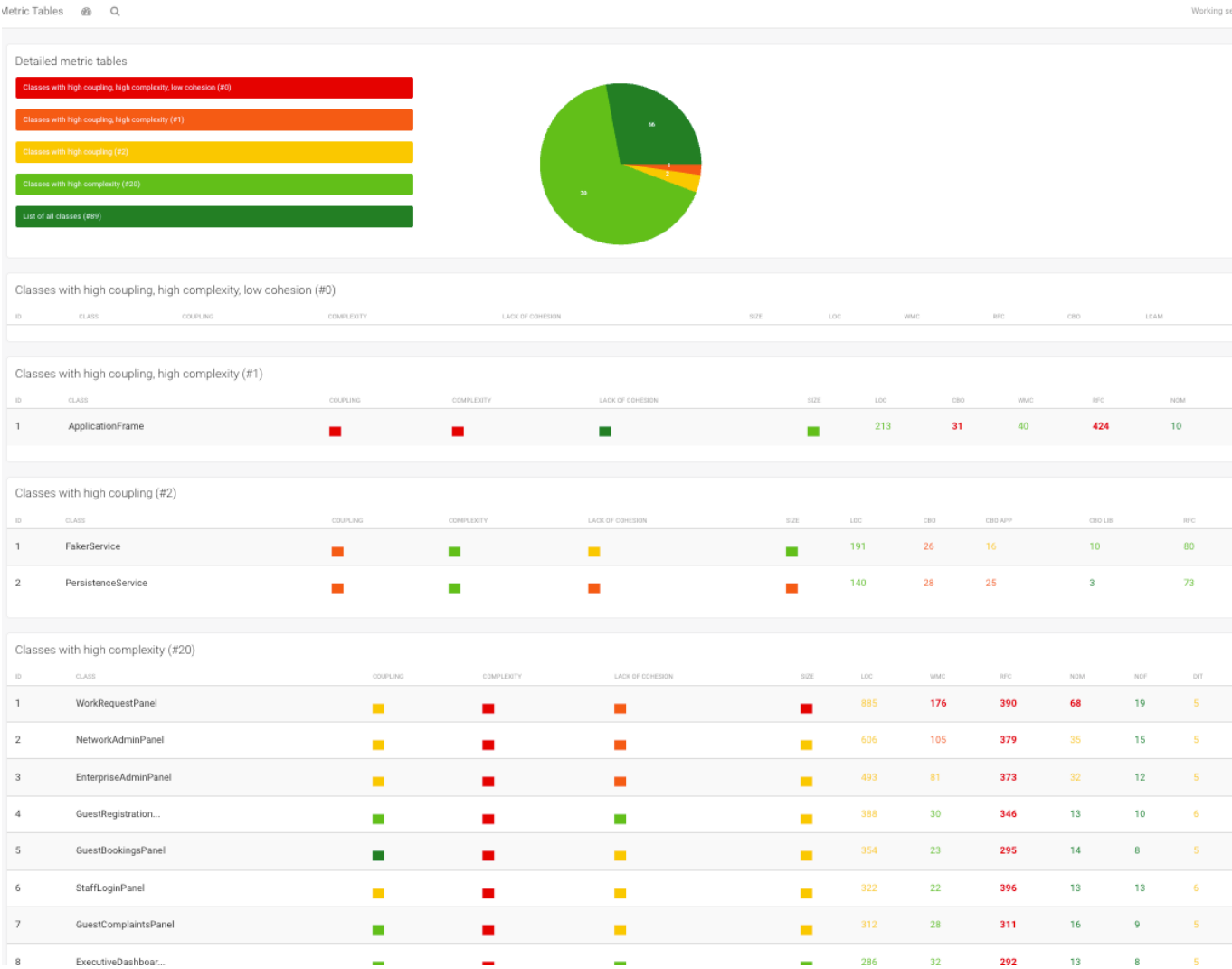
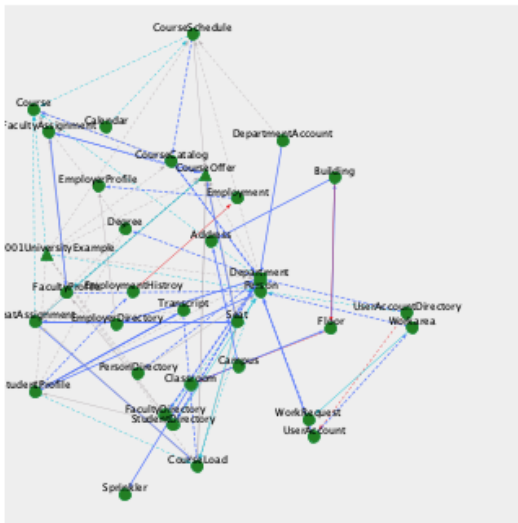


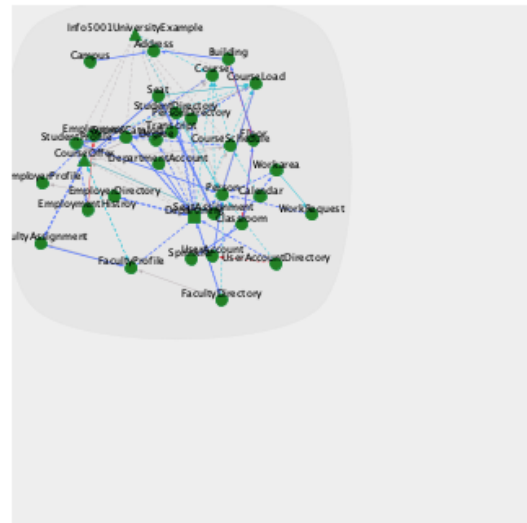
Figure 53. Software Graphs

CodeMR Software Graphs

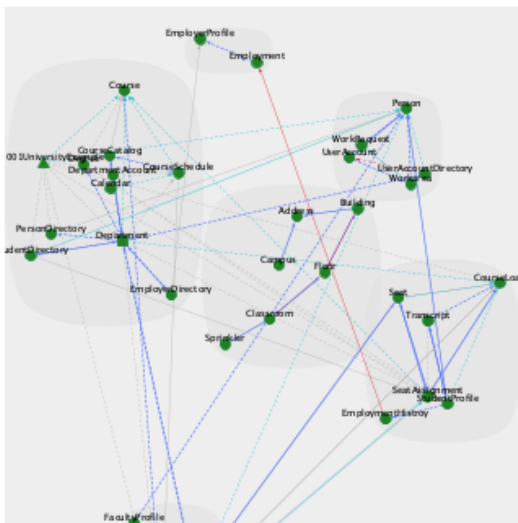
Click on the titles below to enlarge graphs



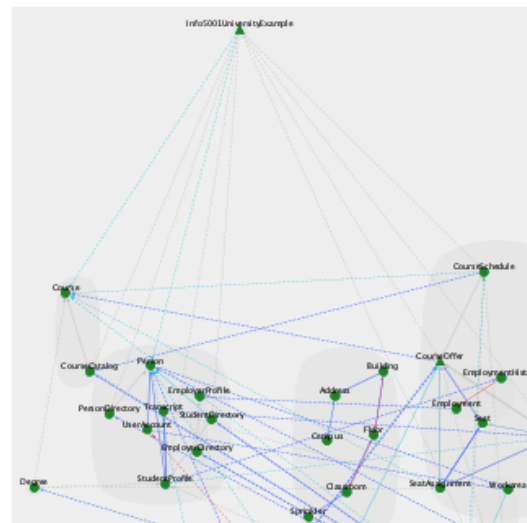
all_nm



all_byprojectmodule



all_bymodularity



all_bypackage

APPENDIX 10: Common Thresholds for Cyclomatic Complexity

Common thresholds		
CYCLOMATIC COMPLEXITY	RISK LEVEL	WHAT IT MEANS
1-10	Low	Simple, well-structured code. Easy to test and maintain.
11-20	Moderate	More complex code. Requires careful testing and should be reviewed for refactoring opportunities.
21-50	High	Difficult to test thoroughly. Likely a candidate for decomposition.
50+	Very High	Very difficult to test thoroughly. Carries elevated defect risk. Review for redesign or decomposition.

Source: Tanner, Matt. 2025. "Cyclomatic Complexity: What it is and how to reduce it." Sourcegraph.com blog. Retrieved from <https://sourcegraph.com/blog/cyclomatic-complexity-what-it-is-and-how-to-reduce-it>

APPENDIX 11: Coupling-Cohesion Quadrant

The coupling-cohesion quadrant

Plotting CBO against TCC creates four natural categories that tell an agent how to approach a class:

- **Healthy** (low coupling, high cohesion): Easy to change, well-designed.
- **Spaghetti** (high coupling, low cohesion): Tangled and internally incoherent. The worst case. Refactor before modifying.
- **Hub** (high coupling, high cohesion): Internally coherent but fragile, because many other classes depend on it. Modify with caution.
- **Island** (low coupling, low cohesion): Isolated but messy inside. Low risk, but could be cleaner.

Source: Schneider, Jonathan. 2026. “Code quality metrics tht AI coding agents can actually use.” Moderne blog. Retrieved from <https://www.moderne.ai/blog/code-quality-metrics-that-ai-coding-agents-can-actually-use>

REFERENCES

CodeMR. (2020, July 25). Documents. Retrieved from <https://www.codemr.co.uk/documents/>

CodeMR. (2021). “Supported Metric List by CodeMR.” Medium blog post. Retrieved from <https://codemr.medium.com/supported-metric-list-by-codemr-a119c3cffc2d>

Schneider, J. (2026). Code quality metrics that AI coding agents can actually use. Moderne blog. Retrieved from <https://www.moderne.ai/blog/code-quality-metrics-that-ai-coding-agents-can-actually-use>

Tanner, M. (2025). Cyclomatic Complexity: What it is and how to reduce it. Sourcegraph.com blog. Retrieved from <https://sourcegraph.com/blog/cyclomatic-complexity-what-it-is-and-how-to-reduce-it>

Methodology and Use of AI Tools

CoPilot and Claude were used to evaluate my draft against the rubric. The analysis, observations, and conclusions reflect the author's own hands-on experience with the tool and the codebases analyzed. All content has been reviewed for accuracy and represents the author's understanding of the results.

Reference to CoPilot conversation:

<https://copilot.microsoft.com/shares/rbaKmoobTLikFTpupSeWJ>

Reference to Claude LLM conversation: [https://claude.ai/share/1510a614-ff10-4195-89b2-](https://claude.ai/share/1510a614-ff10-4195-89b2-eb4e9a474587)

[eb4e9a474587](https://claude.ai/share/1510a614-ff10-4195-89b2-eb4e9a474587)